

Why This Order? Algorithmic Frameworks for Explaining Rankings

Nidhi Jha
IIT Bombay
24d0378@iitb.ac.in

Priyanka Golia
IIT Delhi
pgolia@cse.iitd.ac.in

Sujoy Bhore
IIT Bombay
sujoy@cse.iitb.ac.in

Suraj Shetiya
IIT Bombay
surajs@cse.iitb.ac.in

ABSTRACT

The utility of ranking queries is ubiquitous in database systems and is commonly used in various domains such as job applications, the financial sector, and medical facilities. In the context of algorithmic fairness, a fundamental question emerges: Given a ranking, can we efficiently synthesize linear utility functions that accurately explain it?

In this work, we study four commonly used ranking measures: Pairwise Penalty (PP), Inversions (INV), Minimum Position Error (MPE), and Largest Individual Position Error (LIPE). We design various algorithmic approaches to tackle these problems and develop efficient algorithms. Our methods are based on sophisticated algorithmic frameworks such as Linear Programming, hierarchical space partitioning, and local search. We provide provable guarantees for these methods and conduct extensive experimental evaluations on a variety of datasets, including real-world instances.

PVLDB Reference Format:

Nidhi Jha, Sujoy Bhore, Priyanka Golia, and Suraj Shetiya. Why This Order? Algorithmic Frameworks for Explaining Rankings. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 INTRODUCTION

Ranking is ubiquitous in modern large-scale decision-making systems. Whether we are sorting job applicants by suitability, ordering products by relevance, or prioritizing patients by urgency, most real-world decision systems ultimately boil down to a single ordered list [8, 31]. In large-scale database systems, ranking queries are not only common – they are often the final step before a decision is made [23]. This raises a deceptively simple yet increasingly fundamental question:

*Given a ranking on database tuples without access to the underlying utility rule, can we still explain **why it is ordered this way**?*

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

An affirmative answer would unlock several benefits. For instance, users could interpret the ranking rather than treating it as a black box, or regulators could audit automated decisions for potential bias [6, 33]. On the other hand, database users could debug or refine ranking queries by adjusting the inferred weights [4]. Even when the original ranking is *noisy* or *inconsistent*, having some approximate explanation is far better than having none [4].

Rankings are commonly obtained by aggregating multiple attributes into a single score and ordering tuples accordingly; see e.g., [16, 18, 30, 32]. Among the various scoring paradigms, *linear utility functions* offer a particularly simple and interpretable model, where the score of a tuple is expressed as a weighted combination of its attributes. The weights naturally capture the relative importance of attributes, thereby providing an intuitive explanation of the resulting ranking in terms of user preferences [4, 12, 19, 34]. In addition to their interpretability, linear models are computationally efficient and widely adopted in ranking and recommendation systems as well as in decision-support applications (see [21, 29, 38, 41, 42]). Even when the true ranking function is non-linear or a black-box, similar ideas appear in interpretability methods such as LIME [13, 20], which approximate complex models locally by a linear function (a supporting hyperplane). Thus, our linear model is also applicable as a surrogate for non-linear utility functions, especially for small values of k (i.e., $k \ll n$), where local linearity is a widely used and effective approximation. Motivated by this combination of simplicity, interpretability, and practical relevance, we adopt linear scoring functions as our explanation model.

Unfortunately, turning this intuition into an algorithmic framework is far from trivial. Not every ranking admits an exact linear explanation. Real-world rankings often contain inconsistencies or nonlinear rule-based adjustments (e.g., sponsored boosts [44] or diversity constraints [10]) that no single weight vector can satisfy exactly. Even when approximate explanations exist, the appropriate notion of “distance” between the target ranking and the explained one can vary depending on the application: Should we minimize the number of pairwise disagreements? Or the largest positional shift of any item? Or perhaps the total displacement?

While earlier research has largely focused on *how to generate rankings efficiently* or *how to adapt them to user preferences* [23], our goal is different: we ask whether *existing* rankings can be *explained*. In practice, most database systems implement rankings via *linear utility functions* over item attributes – a paradigm that underlies classical formulations in learning-to-rank [44], linear utility models [37], and recent work on weight inference and query

personalization [4, 12]. However, these works all address the *forward* problem: given attributes, learn a function that generates a desirable ranking. We instead study the *inverse* problem: *given only the ranking, recover an underlying linear utility rule*. More precisely, if an exact linear explanation exists, can we recover it efficiently? And if not, can we compute the *best possible approximation* under a chosen ranking-error metric? Chen et al. [13] initiated this direction, but key algorithmic challenges remain — particularly in high-dimensional settings and under stronger error objectives. These challenges form the core of this work, driven by the needs of *transparency, fairness, and debuggability*.

A core difficulty arises from the high dimensionality of modern ranking models. As the number of attributes grows, the space of feasible weight vectors becomes increasingly complex, and even verifying whether a single linear function is consistent with a given ranking can be computationally demanding. One might hope to apply dimensionality reduction techniques—as is common in nearest-neighbor search or clustering—but rankings are extremely sensitive to distortion: even a small embedding error in feature space can completely reshuffle the order of items. Consequently, classical projection-based methods such as Principal Component Analysis (PCA) [25] or Johnson–Lindenstrauss (JL) embeddings [24] are fundamentally incompatible with ranking preservation. Although margin-based dimensionality reduction techniques [26] provide strong guarantees for learning linear classifiers, they are not directly applicable here: unlike classification, ranking depends on maintaining a precise *relative* order among all items, and even minor perturbations may flip multiple pairwise comparisons.

This makes it necessary to reason about rankings directly in the original high-dimensional space, but in a way that remains algorithmically tractable. Our approach leverages the geometric structure of ranking constraints: each pairwise comparison corresponds to a halfspace inequality over the weight vector, and ranking discrepancies can be interpreted as violations of hyperplane separation. This geometric viewpoint allows us to develop scalable approximation schemes using Linear Programming, hierarchical space partitioning, and local search all of which operate natively in weight space while preserving ranking fidelity.

Problem Overview. Given an input ranking π over tuples in a database $\mathcal{D}$, our goal is to *explain* it using a linear utility function $f_W(x) = W \cdot x$. That is, we seek a weight vector $W = (w_1, \dots, w_d)$ such that the induced ranking ρ_W is as close as possible to π under a chosen “error measure”.

We employ several practically motivated similarity metrics—each designed to capture different aspects of ranking disagreement. PP captures the score difference of tuples that violate the correct top- k ordering, while INV measures the pairwise swaps required to transform the produced ranking into the reference ranking. MPE quantifies the total positional displacement across all tuples, and LIPE reflects the worst individual positional deviation. Together, these measures provide a comprehensive view of how closely the generated ranking aligns with the reference. (see Section 2 for definitions and illustrative examples).

While several methods have been proposed for interpreting ranking decisions, only a few works directly address the problem of deriving explanations in the form of linear utility functions. The

closest prior work [13] studies ranking explanations primarily under the MPE objective and introduces several error measures, including PP, INV, and LIPE, largely treating them independently. In contrast, we consider this broader set of practically meaningful error measures and investigate the formal relationships among them. In particular, in Sections 4.0.2 and 4.0.3, we show how approximation guarantees for one error measure can imply corresponding guarantees for others, providing a unified perspective on ranking explanation quality. Moreover, our methods scale to large datasets and demonstrate improved scalability over [13].

1.1 Summary of Results and Organization.

We present a study of the problem of *explaining an observed ranking using a linear utility function when the underlying model is unknown* (Section 2). A concise summary of our contributions, along with pointers to the relevant sections, is provided below:

- We design a family of practically efficient approximation schemes for the PP objective (Section 3), offering a tunable trade-off between runtime and accuracy. Each variant comes with formal guarantees and is amenable to scalable implementation.
- Next, we establish formal connections between the seemingly different error measures (Section 4), showing that an exact solution under one metric yields approximation guarantees under the others. This provides a unified view of ranking disagreement and enables cross-transfer of algorithmic solutions.
- We develop scalable optimization methods for the INV, MPE, and LIPE objectives (Section 5.1, 5.2). These include a local search framework with update rules, and a hierarchical partitioning strategy (HP-INV) that leverages shifted QuadTrees and LP-based merges to accelerate convergence. Both approaches exhibit strong empirical performance and favorable runtime behavior.
- To increase the efficiency of our LP-based methods, we exploit the structure of top- k rankings to accelerate computation with minimal loss in solution quality (Section 6). Our iterative refinement procedure acts as an *anytime method*, producing an initial solution quickly and improving it as additional time is allocated.
- Our extensive experimental evaluation (Section 7) demonstrates that the proposed methods scale to large datasets and outperform baseline approaches by orders of magnitude in runtime, while maintaining comparable solution quality.

Note about organization: Due to space constraints, proofs of some theorems/lemmas and some pseudo-codes were moved to the appendix which can be accessed in the extended version of our paper¹.

2 PRELIMINARIES

Model: Consider a database $\mathcal{D}$ with n tuples, $\{t_1, t_2, \dots, t_n\}$. Each tuple has d numerical attributes and numerous other (categorical, string) attributes associated with it. These attributes are used for filtering, answering point and range queries, multi-criteria decision making, and so forth. In our work, we focus on explaining rankings over a given set of numerical attributes.

Definition 2.1 (Ranking over a Dataset). Given a database $\mathcal{D} = \{t_1, t_2, \dots, t_n\}$, a *ranking* π is a permutation of the tuples in $\mathcal{D}$ such that adjacent tuples are related by either strict preference or

¹<https://tinyurl.com/lin-rank>

Table 1: Synthetic student scores and ranks (example).

ID	SAT	GRE	TOEFL	Ethnicity	Gender	Rank
C6	75	95	65	C	F	1
C1	82	85	90	B	F	2
C2	88	92	75	A	M	3
C4	90	80	70	A	F	4
C5	70	78	95	B	F	5
C8	92	70	80	B	M	6
C3	85	88	60	C	M	7
C7	60	65	85	A	M	8

equivalence. That is, $\pi = [r_i \succ_1 r_j \succ_2 \dots \succ_{n-1} r_m]$, where each rank corresponding to $t_i \in \mathcal{D}$ is represented as r_i and $\succ_k \in \{>, =\}$ for $k = 1, \dots, n-1$.

In our definition, π is defined over all the tuples of the database $\mathcal{D}$. In practical settings, a user may only be interested in the top- k version of ranking, which we define next.

For any two tuples t_i and t_j , t_i is preferred over t_j if it appears before t_j in the sequence; that is, if $r_i < r_j$. The *rank* of a tuple $t \in \mathcal{D}$ is defined as $\pi(r) = |\{r' \in \mathcal{D} \mid r' > r\}| + 1$. This ranking defines a total order over the dataset, capturing both strict and weak preferences in a structured manner. We define $\mathcal{D}_\pi(k) \subseteq \mathcal{D}$ as the set of top- k tuples according to π , i.e., $\mathcal{D}_\pi(k) = \{t \in \mathcal{D} \mid \pi(t) \leq k\}$.

Running example. Consider a university admissions scenario where applicants are evaluated based on their *SAT*, *GRE*, and *TOEFL* scores. Alongside these numerical attributes, the university collects additional demographic information such as gender and ethnicity. The exact ranking policy, including any usage of Diversity, Equity, and Inclusion (DEI) guidelines, is not disclosed to applicants. Only the final ranking and the numerical attributes are made publicly available. A sample dataset of applicants and their published ranks is shown in Table 1.

We now formally define utility functions and state the problem.

Definition 2.2 (Utility function). A utility function is a mapping $f : \mathcal{D} \rightarrow \mathbb{R}$, which assigns each tuple $t \in \mathcal{D}$ a real-valued score $f(t)$ based on its attribute values. The induced ranking orders tuples in $\mathcal{D}$ in non-increasing order of their scores.

Definition 2.3 (Linear utility function). A linear utility function is specified by a weight vector $W = (w_1, w_2, \dots, w_d) \in \mathbb{R}_{\geq 0}^d$, and maps a tuple $t \in \mathcal{D}$ to the score

$$f_W(t) = \sum_{i=1}^d w_i t[i].$$

In practice, attributes may contribute positively or negatively to the score. For example, large *price* is typically undesirable and hence, lower is better. Such attributes can be negated (i.e., multiplied by -1) so that all contributions are aligned, allowing us to assume non-negative weights without loss of generality. Additionally, following the existing works in ranking [4, 5] we avoid repetitions of utility functions by normalizing the weight vector (i.e. $\|W\|_2 = 1$).

The linear function f_W induces a ranking ρ_W over $\mathcal{D}$ as

$$\begin{aligned} \rho_W(t) = \rho_W(t') &\iff f_W(t) = f_W(t') \\ \rho_W(t) > \rho_W(t') &\iff f_W(t) > f_W(t'). \end{aligned}$$

Similarity measures between rankings. Given a ranking π over a database $\mathcal{D}$, Chen et al. [13] formulate a linear program (LP)

Table 2: Notation

Symbol	Definition
$\mathcal{D}$	Database
$n = \mathcal{D} $	Number of tuples in $\mathcal{D}$
$t[1], \dots, t[d]$	Attributes of $\mathcal{D}$ used for ranking
f_W	Linear utility function over $t[1], \dots, t[d]$
$W = (w_1, \dots, w_d)$	Weight vector defining f_W
$\pi = [r_i \succ_1 r_j \succ_2 \dots \succ_{n-1} r_m]$	Given ranking of $(t_i \in \mathcal{D}, \succ_i \in \{>, =\})$
$\pi(r)$	Rank of $r \in \mathcal{D}$ in π
ρ_W	Ranking of $\mathcal{D}$ induced by function f_W
$\rho_W(r)$	Rank of r based on utility function f_W
$\mathcal{D}_\pi(k)$	Top- k result for ranking π
$\mathcal{D}_W(k)$	Top- k result for utility function f_W
s_{ij}	Slack variable for error of tuples $t_i, t_j \in \mathcal{D}$

that attempts to recover a linear utility function whose induced ranking ρ_W exactly matches π . When no such function exists, the natural goal is to find a utility function whose ranking over $\mathcal{D}$ is as *similar* as possible to π , so that its weights can be interpreted as an explanation of feature importance. In this work, we adopt the error measures introduced in [13] to quantify this similarity; these are summarized below.

- **Pairwise penalty (PP):** An insightful interpretation of the PP error is that it represents the minimum total adjustment required in the scores so that the induced ranking matches π exactly. Consequently, the PP value of the optimal utility function corresponds to the smallest modification (over all linear models) needed to transform the scores into one that yields π . Formally,

$$\sum_{\{r \in \mathcal{D}_\pi(k), s \in \mathcal{D}, \pi(r) < \pi(s), |W|_2=1 \mid \rho_W(r) > \rho_W(s)\}} f_W(s) - f_W(r) \quad (1)$$

Beyond PP, Chen et al. [13] also propose *three closely related variants*, which capture alternative but complementary perspectives on disagreement between rankings. These measures are equally meaningful in practice and are often useful depending on whether one prefers counting violations or measuring positional displacement.

- **Inversions (INV):** The INV measure captures how many pairwise exchanges of tuple positions are necessary to convert the generated ranking into the reference ranking π .

$$|\{r \in \mathcal{D}_\pi(k), s \in \mathcal{D}, \pi(r) < \pi(s) \mid \rho_W(r) > \rho_W(s)\}| \quad (2)$$

- **Minimum position error (MPE):** MPE measures the total amount of displacement of tuples in the generated ranking compared to their positions in the reference ranking π .

$$\sum_{r \in \mathcal{D}_\pi(k)} |\rho_W(r) - \pi(r)| \quad (3)$$

- **Largest individual position error (LIPE):** LIPE captures the maximum positional deviation of any tuple between the generated and the reference ranking π . It highlights the worst-affected tuple - the one ranked farthest from where it should be.

$$\max_{r \in \mathcal{D}_\pi(k)} |\rho_W(r) - \pi(r)| \quad (4)$$

To illustrate the four error measures, consider the top-3 tuples in π : C6, C1, and C2, as shown in Table 1. Since ranking π is not realizable by any linear function, our objective is to find a linear utility function whose induced ranking is as similar to π as possible. Consider the linear function $W_1 = (0.13, 0.95, 0.27)$, which

induces the ranking [C2, C6, C1, C3, C5, C4, C8, C7] with scores [119.97, 118.30, 116.53, 111.70, 109.55, 107.50, 100.98, 93.10]. Under this ranking, C1 and C2 change positions relative to π , contributing $f(C2) - f(C1) = 3.44$ to the PP error. Similarly, C6 falls below C2, adding $f(C2) - f(C6) = 1.67$. Thus, the total PP error is 5.11, and the number of INV violations is 2. The MPE value is 4, since C6 and C1 move by one position each and C2 moves by two positions. Finally, LIPE is 2, as the maximum displacement of any tuple is two ranks.

Note on PP error: Since PP depends on score differences, its value scales with the magnitude of W . To avoid arbitrary scaling, we normalize the weight vector by enforcing $\|W\|_2 = 1$ following existing works in ranking literature [4, 5]. Under the lack of such constraints, scaling the weight vector to near 0, produces a ranking with a PP error of 0.

2.1 Problem Formulation

Our objective is to explain an observed ranking π by finding a linear utility function f_W whose induced ranking ρ_W is as similar to π as possible. We formalize this below in a similarity-agnostic manner.

Given a database $\mathcal{D}$ of tuples, an input ranking π over $\mathcal{D}$, and a chosen error measure (as defined above), find a linear utility function f_W with weight vector $W = (w_1, w_2, \dots, w_d)$ such that the error between its induced ranking ρ_W and π is minimized.

Among the error measures, we place particular emphasis on Pairwise Penalty, as it captures both score differences and rank discrepancies in a balanced manner. Accordingly, the first part of this paper focuses on developing scalable methods for the PP objective, formally defined below.

PP Error: Given a database $\mathcal{D}$ of tuples and an input ranking π over $\mathcal{D}$, compute a linear utility function f_W with weight vector $W = (w_1, w_2, \dots, w_d)$ such that the PP error between its induced ranking ρ_W and π is minimized.

3 ALGORITHMIC APPROACHES FOR PP ERROR

In this section, we describe our methods that optimize for the PP error-measure. Our approach exploits this score-based formulation to formulate a Linear Program (LP) for this problem.

3.1 LP formulation for PP error

Our LP formulation introduces d variables for the weight vector and $\binom{n}{2}$ slack variables for error, i.e. s_{ij} for every pair of items (t_i, t_j) . The variable s_{ij} captures the error incurred when t_j is ranked better than t_i by the ranking function while t_i is ranked better than t_j in π . Variable s_{ij} becomes 0 otherwise. These slack variables can be broadly categorized as follows:

- (1) **Order violations within the ranking.** If an item that appears lower in the target ranking π receives a score higher than an item above it, the slack variable captures the error (difference of the scores).
- (2) **Tie violations.** If two items designated as tied in π receive different scores under f_W , a slack variable is added to penalize this deviation.

- (3) **Top- k versus outside violations.** If any item outside the top- k receives a score higher than an item within the top- k , the violation is penalized by an associated slack variable.

The overall LP thus enforces ranking consistency as much as possible, while allowing controlled deviations where no feasible weight vector exists that perfectly realizes π . The optimization objective is given by: $\min \sum_{i,j} s_{ij}$, where each s_{ij} corresponds to the slack variables described in the above violations.

To handle the three kinds of violations, we add constraints for every s_{ij} as below,

$$\begin{aligned} \forall \{i, j | \pi_i \leq \pi_j\} \quad s_{ij} &\geq 0 \\ \forall \{i, j | \pi_i \leq \pi_j\} \quad s_{ij} &\geq \sum_{k=1}^d w_k t_j[k] - w_k t_i[k] \end{aligned}$$

An important restriction on the weight vector is to restrict the set of weight vectors to unit length. But, such a constraint $\sum_{k=1}^d w_k^2 = 1$, renders our linear program non-convex. Hence, we instead replace this constraint by the affine constraint $\sum_{k=1}^d w_k = 1$. We analyze the effect of replacing the ℓ_2 norm by the ℓ_1 norm as a part of Theorem 3.1. The LP formulation is given in Equation 5.

$$\begin{aligned} \text{minimize} \quad & \sum_{i \in \mathcal{D}_\pi(k)} \sum_{j=0}^n s_{ij} \\ \text{s.t.} \quad & \forall \{i, j\} \quad s_{ij} \geq 0 \\ & \forall \{i, j \in \mathcal{D}_\pi(k) \mid \pi_i < \pi_j\} \quad s_{ij} \geq \sum_{k=1}^d w_k t_j[k] - w_k t_i[k] \\ & \forall \{i, j \in \mathcal{D}_\pi(k) \mid \pi_i = \pi_j\} \quad s_{ij} \geq \sum_{k=1}^d w_k t_j[k] - w_k t_i[k] \\ & \forall \{i, j \in \mathcal{D}_\pi(k) \mid \pi_i = \pi_j\} \quad s_{ij} \geq \sum_{k=1}^d w_k t_i[k] - w_k t_j[k] \\ & \forall \{i \in \mathcal{D}_\pi(k), j \notin \mathcal{D}_\pi(k)\} \quad s_{ij} \geq \sum_{k=1}^d w_k t_j[k] - w_k t_i[k] \\ & \forall k \in \{1, \dots, d\} \quad w_k \geq 0 \\ & \sum_{k=1}^d w_k = 1 \end{aligned} \tag{5}$$

For example 1, instantiating the PP formulation yields a strictly positive objective value and returns the weight vector $\mathbf{w} = (0.30, 0.42, 0.27)$ that minimizes total slack in the ordering constraints. The induced ranking [C2, C1, C6, C5, C4, C3, C8, C7] differs from the input order by moving C2, C1 above C6 and swapping (C4, C5), (C1, C2) and (C8, C3), demonstrating the minimum relaxation required to obtain a realizable linear ranking. Our method computes a linear utility function normalized under the ℓ_1 norm, achieving an objective value of 10.81. When the weight vector is rescaled to have unit ℓ_2 norm, the corresponding PP error becomes 18.38. Under the same ℓ_2 normalization, the optimal value is 17.916.

We formally establish both the efficiency and accuracy of our approach in Theorem 3.1.

THEOREM 3.1. *Given a database $\mathcal{D}$ and a ranking order π , let OPT^* denote the minimum possible PP error over all linear utility functions. Then the linear program in Equation 5 returns a solution whose error is at most $\sqrt{d} \cdot \text{OPT}^*$. Moreover, it can be solved in time $O((n^2 + d)^{2.5}L)$, where L is the number of bits required to represent the input.*

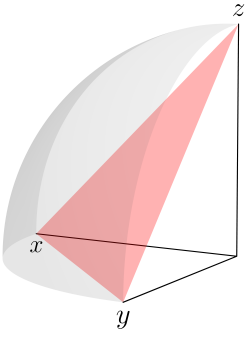


Figure 1: The plane $|W|_1 = 1$ which approximates the non-convex spherical surface $|W|_2 = 1$ is shown in pink.

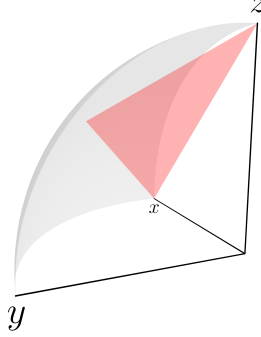


Figure 2: Part of the function space W approximated by the plane passing through $(1, 0, 0)$, $(0, 0, 1)$ and $(\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}})$

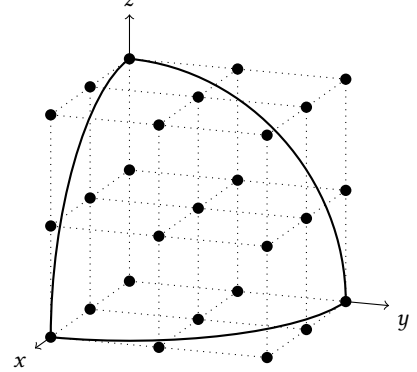


Figure 3: Function space W split into grids with an $\delta = 0.5$

PROOF. The proof proceeds in two parts: we first establish the approximation guarantee, and then analyze the running time.

Approximation analysis: Our approach simulates the surface $\|W\|_2 = 1$ by $\|W\|_1 = 1$ to convert the non-convex optimization problem into a convex one. Our analysis relies on this simulation to prove the approximation bounds. To analyze our approach, we reformulate the optimization problem using vector notation and then proceed to prove the approximation guarantee. Consider the optimization function that corresponds to the weight functions on the surface $\|W\|_2 = 1$. Note that each of the summations $\sum_{k=1}^d w_k t[k]$ in our LP can be written in the form $W \cdot t$, where W and t correspond to vectors. Thus, the inequalities in Equation 5 can be interpreted as *dot products*.

We are now ready to prove the approximation guarantee. We make use of the property that $W \cdot t = \|W\|_2 \|t\|_2 \cos(\theta)$, where θ is the angle between the vectors W and t . As the optimization problem in Equation 5 uses the ℓ_1 norm instead of the ℓ_2 norm in the summation (i.e., $\|W\|_1 \|t\|_2 \cos(\theta)$), the error incurred is at most $\frac{\|W\|_1}{\|W\|_2} \cdot \text{OPT}^*$. The value of $\frac{\|W\|_1}{\|W\|_2}$ is minimized for the weight vector $(\frac{1}{\sqrt{d}}, \frac{1}{\sqrt{d}}, \dots, \frac{1}{\sqrt{d}})$. Thus, the least value produced by the LP in Equation 5 is $\frac{1}{\sqrt{d}} \cdot \text{OPT}^*$.

Runtime analysis: Given an LP with N variables and M constraints, the time complexity of solving it is $(N + M)^{1.5} NL$ (see Vaidya [39]), where L is the number of bits needed to represent the input. The LP in Equation 5 has $k^2 + nk + d = O(nk + d)$ variables and $O(nk + d)$ constraints, which gives us a running time of $O((nk + d)^{2.5} L)$. In the worst case $k \in O(n)$, then $\binom{n}{2} + d = O(n^2 + d)$ variables and $O(\binom{n}{2} + d)$ constraints. Thus, the overall running time of our approach is $O((n^2 + d)^{2.5} L)$. Thereby, we conclude the proof of the theorem. $\square$

Note on LP solvers: Our running-time bound relies on Vaidya’s algorithm [39]. More recent progress by Cohen et al. [14] achieves a running time of $\tilde{O}(n^\omega)$, where $\omega \approx 2.37$ is the current matrix-multiplication exponent [2]. Any further improvements in LP solvers would immediately translate to faster runtimes for our algorithm. We also note that, in practice, the Simplex method often outperforms theoretically faster worst-case algorithms [15, 36].

3.2 Fine-Grained LP Refinements

We propose two fine-grained variants of the LP-PP approach by carefully partitioning the function space and running an LP on each part. If it were possible to include the constraint $\|W\|_2 = 1$ directly in the LP, we could compute the exact error. However, since this constraint is non-convex, we instead “simulate” the spherical surface $\|W\|_2 = 1$ using multiple piecewise linear surfaces. Increasing the number of such linear surfaces improves accuracy, but at the cost of running time. We first present a simpler refinement of the LP, followed by a polynomial-time approximation scheme (PTAS).

3.2.1 Fixed-Pivot Refinement. The first fine-grained version builds directly on the analysis of LP-PP in Theorem 3.1. In that proof, we identified that the worst-case behavior of LP-PP occurs when the weight vector is $(\frac{1}{\sqrt{d}}, \frac{1}{\sqrt{d}}, \dots, \frac{1}{\sqrt{d}})$. We refer to this point as the *pivot point*. To improve the approximation factor, we approximate the non-convex spherical surface $\|W\|_2 = 1$ using d distinct linear surfaces.

As a first step toward this refined simulation, we partition the function space uniformly into d regions. The partition is centered around the pivot point, since the weights near this point form the “Achilles’ heel” of LP-PP (see Theorem 3.1). To describe the partitioning scheme in more detail, consider the hyperplane $\|W\|_1 = 1$, which “simulates” the surface $\|W\|_2 = 1$. This hyperplane is valid under the constraint $w_i \geq 0$ for all i , and therefore has d corner points defining its boundary. Our idea is to partition this space into d regions by replacing one of these corner points with the *pivot point*. Each such replacement yields a distinct piecewise-linear approximation of the spherical surface $\|W\|_2 = 1$.

We then solve d separate LPs, each analogous to the original formulation in Equation 5, but with the constraint now restricted to one of the pivot-based hyperplanes instead of $\|W\|_1 = 1$. Each LP returns the error for its respective surface, and we aggregate these by taking the min over all d values. We refer to this partitioning-based refinement as dP-PP.

To illustrate the idea in three dimensions, consider the surface shown in Figure 1, which depicts the hyperplane $\|W\|_1 = 1$ used to approximate the spherical surface $\|W\|_2 = 1$. To obtain a finer partition of the function space, we insert the point $(\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}})$

and combine it with each pair of corner points $(1, 0, 0)$, $(0, 1, 0)$, and $(0, 0, 1)$, thereby generating three new planes. One of these planes is shown in Figure 2. For our running example in table 1, the dP-PP algorithm returns the weight vector $(0.267, 0.481, 0.252)$, with the corresponding PP error 17.92. The corresponding ℓ_2 norm weights are $(0.441, 0.795, 0.416)$ which gives a overall error of 17.92.

We formally establish both the running time and the improved approximation guarantee in Theorem 3.2.

THEOREM 3.2. *Given a database $\mathcal{D}$ and a ranking order π , let OPT^* denote the optimal PP error. Then dP-PP returns a solution with error at most $\frac{1}{\sqrt{d^2 - 2\sqrt{d}(d-1)}} \cdot \text{OPT}^*$, and runs in time $O(d(n^2 + d)^{2.5}L)$, where L is the number of bits required to represent the input.*

PROOF SKETCH. We analyze both the approximation guarantee and running time of dP-PP.

Approximation bound. By symmetry, it suffices to analyze one of the d partitions. The corresponding hyperplane (see Equation 9) is given by $w_1 + \dots + w_{d-1} - (d-1-\sqrt{d})w_d = 1$. Following the argument from Theorem 3.1, the approximation factor is the reciprocal of the Euclidean distance from the origin to this hyperplane, which equals $1/\sqrt{(d-1) + (d-1-\sqrt{d})^2} = 1/\sqrt{d^2 - 2\sqrt{d}(d-1)}$. Hence, dP-PP incurs error at most $\frac{1}{\sqrt{d^2 - 2\sqrt{d}(d-1)}} \cdot \text{OPT}^*$.

Running time. dP-PP invokes LP-PP d times, so its total runtime is $O(d(n^2 + d)^{2.5}L)$. $\square$

Due to a lack of space, we move the proof to the Appendix A.

3.2.2 Hierarchical LP Refinement. Inspired by the fine-grained dP-PP result, we construct a hierarchical partitioning of the space to obtain a tunable trade-off between running time and approximation quality. We treat dP-PP as the first level of this hierarchy, producing d initial regions. For each such region, we again perform a decomposition around its corresponding pivot point, i.e., the weight vector within that region that minimizes the function length. Continuing this process recursively, level l consists of d^l regions in total. We present our approach in Algorithm 1.

Our solution hinges on efficiently identifying the pivot point. Although the analysis of dP-PP in Theorem 3.2 relies on the tangent construction to derive the pivot point analytically, we have observed experimentally that this bound is loose. In particular, the intersection of the tangent with the unit d -sphere $\sum_{k=1}^d w_k^2 = 1$ often lies outside the positive orthant, and therefore cannot be used directly as a valid pivot point for optimization.

To address this, we formulate a Quadratic Program (QP), given in Equation 6, to compute the pivot point explicitly. The QP is convex, since its Hessian is the $d \times d$ identity matrix, which is positive semi-definite. Hence, the problem can be solved efficiently.

$$\begin{aligned} & \text{minimise} && \sum_{i=1}^d w_i^2 \\ & \text{s.t.} && \sum_{i=1}^d a_i w_i = c \\ & && w_i \geq 0 \quad \forall i \in [d] \end{aligned} \quad (6)$$

Building on the convex QP formulation and the dP-PP refinement, we design a hierarchical partitioning strategy, presented in Algorithm 1.

Algorithm 1: Hierarchical Partitioning Approach for PP

Input: Database $\mathcal{D}$, ranking π , error parameter ϵ

Output: Realizable ranking ρ with error at most $\epsilon \cdot \text{OPT}^*$

```

1 regions  $\leftarrow$  initial region defined by the corner points of the
   positive orthant of the unit  $d$ -sphere
2 errorPP  $\leftarrow \infty$ ;  $\rho \leftarrow \emptyset$ 
3 while regions  $\neq \emptyset$  do
4   region  $\leftarrow$  pop one region from regions
5   Solve dP-PP on region to obtain weights  $w'$ ,  $\rho'$  and error  $e$ 
6   if  $e < \text{errorPP}$  then
7     errorPP  $\leftarrow e$ ;  $\rho \leftarrow \rho'$ 
8     Find pivot  $p$  for region using the QP from Equation 6
9     if  $\frac{e}{\|p\|_2} \geq \epsilon$  then
10      Append  $d$  new regions formed by partitioning around
         $p$  into regions
11 return  $\rho$ 
```

We conjecture that the approximation factor improves monotonically with each refinement level and that a level- l decomposition admits a strictly better bound than single-level dP-PP. Establishing a formal guarantee for this hierarchy presents an exciting direction for further investigation.

3.2.3 Accuracy-Efficiency Trade-off Scheme. We employ a grid-based partitioning of the weight space W , effectively *simulating* the non-convex surface $\|W\|_2 = 1$ using a collection of hypercubes. Rather than working directly with the positive portion of the unit d -sphere, we embed it into the unit hypercube $[0, 1]^d$, which fully contains all feasible ranking functions. This discretization enables a tunable balance between approximation quality and running time via the choice of grid resolution.

The procedure begins by partitioning the unit hypercube $[0, 1]^d$ into grid cells of side length δ . Each such cell is defined by its corner points, forming a δ -sided hypercube in d -dimensional space. For every δ -hypercube that lies entirely within the unit d -sphere, we solve a modified version of the LP from Equation 5. Instead of enforcing the constraint $\sum_{k=1}^d w_k = 1$, we restrict each weight to the corresponding interval $l_i \leq w_i \leq u_i$ for all $i \in [d]$. The final output is obtained by selecting the minimum error value across all such LPs.

The grid resolution δ is crucial for obtaining a PTAS. To ensure an error of at most ϵ , we set $\delta = \frac{\epsilon}{\sqrt{d(1+\epsilon)}}$. We later prove that this yields a *Pairwise Parity* solution with error at most $(1 + \epsilon) \text{OPT}^*$. As illustrated in Figure 3, the unit cube is partitioned into δ -cubes, and the modified LP from Theorem 3.1 is run only on those fully contained within the unit sphere. We refer to this refinement as the *Grid-based Pairwise Penalty* (GPP) for the remainder of the paper.

THEOREM 3.3. *Given a database $\mathcal{D}$ and a ranking order π , let OPT^* denote the optimal PP error. Then the GPP method returns a solution with error at most $(1 + \epsilon) \text{OPT}^*$ and runs in time $O\left(\frac{(\pi e/2)^{d/2}}{\epsilon^d} (n^2 + d)^{2.5}\right)$.*

Due to a lack of space, we move the proof to the Appendix A.

4 INTEGER LINEAR PROGRAMMING FORMULATIONS

In this section, we propose integer linear programming (ILP) formulations for the three other problem variants INV, MPE, and LIPE. We further demonstrate relationships among these error measures, a phenomenon also reflected in our experimental results (Section 7).

4.0.1 ILP formulation for INV. The inversion-based variant compares the input ranking π with the ranking ρ_W induced by the scoring function f_W , and penalizes *inversion* between them. Formally, the error is defined as the minimum number of pairwise exchanges required to transform ρ_W into π . Our ILP formulation for this variant is inspired by the modeling techniques introduced in [13], which modeled the MPE error measure via an ILP formulation.

Our ILP formulation introduces $\binom{n}{2}$ binary variables. Each variable E_{ij} indicates whether an exchange is required between tuples t_i and t_j in ρ_W to obtain π . Moreover, E_{ij} must accurately reflect such exchanges for all valid choices of W . The optimization objective is therefore given by minimise $\sum_{i=1}^{n-1} \sum_{j=i+1}^n E_{ij}$. We start with this simple observation that an exchange between t_i and t_j is needed only when their ranks are swapped in ρ_W . Thus, the penalty should only be applicable under this condition.

Consider the scenario where tuple t_i is ranked better than t_j in π . The penalty for E_{ij} can be represented as $\sum_{k=1}^d w_k t_i[k] \geq w_k t_j[k] + \delta E_{ij}$. The objective is valid under the assumption that an appropriate value of δ is chosen. One simple way to select δ is to compute the ℓ_2 norm of every tuple $t_i \in \mathcal{D}$ and take the maximum among them. This is sufficient because $W \cdot t_i$ cannot exceed this value. With this choice of δ , the complete ILP is as follows.

$$\begin{aligned} & \text{minimize} && \sum_{i=1}^{n-1} \sum_{j=i+1}^n E_{ij} \\ & \text{s.t.} && \forall \{i, j \mid \pi(t_i) < \pi(t_j)\} : \\ & && \sum_{k=1}^d w_k t_i[k] + \delta E_{ij} \geq \sum_{k=1}^d w_k t_j[k] \\ & && \forall k \in \{1, \dots, d\} : w_k \geq 0 \\ & && \forall i, j : E_{ij} \in \{0, 1\} \\ & && \sum_{k=1}^d w_k = 1 \end{aligned} \quad (7)$$

An interesting aspect of this ILP is that constraining W using the ℓ_1 norm is sufficient, unlike in the Pairwise Penalty formulation. This is because PP depends on the magnitudes of score differences, whereas the inversion count only depends on the relative ordering and is therefore insensitive to scaling.

4.0.2 Connection between PP and INV error measures. A key observation in our formulation is that relaxing the ILP for the inversion-based measure (Equation 7) yields exactly the LP of the Pairwise Penalty problem (Equation 5). However, to obtain an optimal solution under this relaxation, the weight vector W must be normalized using the ℓ_2 norm rather than the ℓ_1 norm. This highlights a fundamental distinction: while INV is invariant under scaling, PP is sensitive to score magnitudes and therefore requires ℓ_2 normalization.

4.0.3 Connection between MPE and INV error measures. Using Equation 3, we can upper bound MPE for any ranking over a dataset $\mathcal{D}$ of n tuples as follows. Each tuple contributes at most the

maximum possible rank displacement. This occurs when the tuple in position i in π moves to position $(n - i + 1)$ in ρ_W , i.e., when the ranking is completely reversed. Thus,

$$\text{MPE} \leq 2((n-1) + (n-3) + \dots + 1).$$

In contrast, the INV measure counts the number of inverted pairs (r, s) such that $\pi(r) < \pi(s)$ but $\rho_W(r) > \rho_W(s)$. Since there are $O(n^2)$ such pairs in the worst case, we have $\text{INV} = O(n^2)$.

This shows that both MPE and INV grow quadratically in the worst case, suggesting a strong relationship between the two error measures.

LEMMA 1. *For any input ranked order π and the ranked order produced by utility function f_W , $\text{INV} \leq \text{MPE} \leq 2 \times \text{INV}$*

Due to a lack of space, we move the proof to the Appendix B. **Observation.** For any input ranking π and ranking ρ_W induced by the utility function f_W , the following bounds hold:

$$\text{LIPE} \leq \text{MPE} \leq n \cdot \text{LIPE}, \quad \text{LIPE} \leq 2 \cdot \text{INV} \leq 2n \cdot \text{LIPE}.$$

5 EFFICIENT ALGORITHMIC APPROACHES

In the previous section, we established connections between different error measures. We now develop efficient optimization methods for the INV, MPE, and LIPE objectives. Our approaches combine local search with hierarchical space partitioning (originally designed for INV), and naturally extend to the other two metrics since improving INV also reduces MPE and LIPE.

5.1 Local-Search Ranking

We propose a local-search strategy that iteratively transforms a feasible ranking into a neighboring one that improves the error measure. The process continues until no adjacent ranking yields improvement. We begin by defining the neighborhood structure on rankings.

Definition 5.1 (Neighborhood of a ranking). Two rankings β and γ are considered neighbors if the two orders differ by a single inversion.

For instance, the ranking order $[t_1, t_2, t_4, t_3]$ is considered a neighbor of $[t_1, t_2, t_3, t_4]$, since t_3 and t_4 , which were adjacent, have swapped positions.

Our Local-Search algorithm takes as input a starting ranking order that is realizable by a linear utility function. At each step, it examines neighboring ranking orders and checks (i) feasibility - i.e., whether the neighbor is realizable by some linear utility function - and (ii) whether it improves the error score. If such a neighbor exists, it is accepted as the new current ranking. The algorithm continues applying such improvements iteratively until no neighboring ranking remains that is both feasible and yields a lower error score. At this point, the process terminates at a local minimum.

The pseudo-code for Local-Search is provided in Algorithm 2. Feasibility is verified using the LP formulation in Equation 8.

Algorithm 2: LS-INV: Local-Search for Inversion

Input: Ranking order π , Database $\mathcal{D}$, initial ranking ρ'
Output: Ranking order ρ corresponding to a local minima

```
1  $\rho \leftarrow \rho'$ 
2 while  $\rho$  is changing do
3   foreach  $\rho' \in \text{neighborhood}(\rho)$  do
4     if  $\text{INV}(\rho') < \text{INV}(\rho)$  and  $\text{FEASIBLE}(\rho')$  then
5        $\rho \leftarrow \rho'$ 
6 return  $\rho$ 
```

$$\begin{aligned} & \text{minimize } \sum_{i=1}^{n-1} s_{i,i+1} \\ & \text{s.t. } s_{i,i+1} \geq 0 \quad \forall i : \pi_i < \pi_{i+1}, \\ & \quad s_{i,i+1} \geq \sum_{k=1}^d w_k (t_{i+1}[k] - t_i[k]) \quad \forall i : \pi_i < \pi_{i+1}, \quad (8) \\ & \quad w_k \geq 0 \quad \forall k \in [d], \\ & \quad \sum_{k=1}^d w_k = 1 \end{aligned}$$

In each iteration, algorithm 2 makes progress by decreasing the chosen error measure (INV, MPE, or LIPE). Since this measure is bounded below by 0 and is reduced by at least a fixed positive amount in every iteration, it cannot decrease indefinitely. Consequently, there must exist a point at which no neighboring ranking yields further improvement. At that point, the termination condition is met and the algorithm halts. Thus, the procedure is guaranteed to terminate after a finite number of steps. We prove per-iteration time complexity by Algorithm 2 in Theorem 5.2.

THEOREM 5.2. *Given a database $\mathcal{D}$ and an input ranking π , one iteration of Algorithm 2 can be executed in time $O(n(n+d)^{2.5}L)$, where L denotes the number of bits required to represent the input.*

PROOF. For any given ranking order, there are at most $n - 1$ neighboring rankings obtained by swapping adjacent elements. For each such neighbor, the algorithm performs a feasibility check by solving a linear program. Each LP involves $O(n + d)$ variables and constraints, and thus can be solved in $O((n + d)^{2.5}L)$ time using standard interior-point bounds.

Since up to $n - 1$ such feasibility checks are performed in one iteration, the total time per iteration is $O(n \cdot (n + d)^{2.5}L)$, where L denotes the number of bits required to represent the input. $\square$

For the example in table 1, the LS-INV algorithm converges to a locally optimal ranking [C2, C1, C6, C4, C5, C8, C3, C7] differing from the target order by three pairwise inversions, thus capturing the minimum ordering changes required under the INV measure.

Note on overall runtime. For both INV and MPE, the error decreases by 1 in each iteration and is initially at most $O(n^2)$, yielding at most $O(n^2)$ iterations. Combined with the per-iteration cost from Theorem 5.2, the total running time is $O(n^3(n + d)^{2.5}L)$. For LIPE, the error lies in $[0, n]$ and decreases monotonically in a manner analogous to INV, so the same bound applies.

Initial ranking. As with any hill-climbing or local-search method, the choice of the starting point is crucial. A naive strategy would be to enumerate potential rankings until a feasible one is found. However, since there are $n!$ possible orderings-many of which are not even realizable by a linear utility function-such an approach is

impractical. A practical alternative is to sample s random weight vectors from the function space, compute their induced rankings and corresponding error scores, and select the best among them as the starting point for local search. While this strategy does not provide an unbiased sampler over the space of all feasible rankings, it offers a fast and effective way to obtain a strong initial solution.

Connection between optimization objectives. An important property of our Local Search framework is that minimizing the INV error implicitly reduces both MPE and LIPE. Consequently, any local optimum with respect to INV is also a local optimum under these two measures.

5.2 Hierarchical Partitioning for Inversions

A scalable geometric strategy for the INV objective follows a divide-and-conquer philosophy. Viewing each tuple as a point in $\mathbb{R}^d$, nearby points tend to receive similar scores under any linear function, making it difficult to reproduce their precise relative order in π . To address this, we construct feasible utility functions locally over spatially coherent subsets, and then merge these partial solutions in a bottom-up manner while preserving feasibility.

To describe our approach, we first introduce the notion of a projected ranking.

Definition 5.3 (Projected ranking). Given a ranking π over the tuples of the database $\mathcal{D}$ and a subset $S \subseteq \mathcal{D}$, the projection π_S is the order induced on S by π , i.e., the tuples of S arranged according to their positions in π .

To partition the space, we employ a hierarchical spatial decomposition of $\mathcal{D}$. Specifically, we use a shifted quad-tree [3], which is guaranteed to remain balanced by design. The structure is recursively refined until each leaf cell contains a single tuple.

For the *conquer* phase, we process the tree bottom-up, storing realizable rankings at each node. Since every leaf contains a single point, any linear function trivially satisfies it, providing the base case. At each internal node, rankings from its children are merged to construct a feasible ordering for their union. See Algorithm 4 in Appendix C for the complete pseudo-code of the procedure.

Merge step. Let L_1 and L_2 be the realizable rankings stored at two sibling nodes. The goal of the merge step is to construct a feasible ranking over $L_1 \cup L_2$. A simple greedy strategy is to fix the order of L_1 and insert tuples from L_2 one at a time. For each tuple $t_i \in L_2$, we attempt to place it in each of the $|L_1| + 1$ possible positions within L_1 , test feasibility using the LP in Equation (8), and accept the placement that minimizes the INV error over $\pi_{L_1 \cup \{t_i\}}$.

We perform this merge in two symmetric ways: (i) fixing L_1 and inserting elements from L_2 , and (ii) fixing L_2 and inserting elements from L_1 . The ordering with the smaller INV error is retained. Starting from the leaves, we apply this merge recursively up to the root. The merge procedure is detailed in Algorithm 3, and the full HP-INV algorithm appears in Algorithm 4, both of which are moved to the Appendix A. While this approach does not provide an explicit error guarantee, its runtime for the INV objective is formally analyzed in Theorem 5.4.

THEOREM 5.4 (TIME COMPLEXITY). *The overall running time of the hierarchical partitioning algorithm is $O(n^3(n + d)^{2.5}L)$, where L denotes the number of bits required to encode the input.*

Due to a lack of space, we move the proof to Appendix C.

For the running example in table 1, the HP-INV algorithm 4 produces a realizable ranking by constructing and merging locally feasible rankings and extracting a linear utility function from the final ordering. The resulting ranking [C6, C2, C1, C3, C4, C5, C8, C7] differs from the target ranking by three pairwise inversions.

Practical considerations. To improve scalability, we restrict the quad-tree construction and divide-and-conquer procedure to only the top- k tuples. Furthermore, rather than performing a full feasibility-based merge as in Algorithm 3, we employ a lightweight heuristic that selects a function consistent with the top- k ordering, without explicitly checking feasibility over the remaining tuples.

6 EFFICIENT OPTIMIZATION OF THE LP

We optimize the base LP in Equation 5 by introducing two targeted modifications that significantly reduce running time, while preserving control over the original objective.

(A) Minimum among Top- k vs. Outside

Pairwise constraints between each outside item and all k items in the top set are computationally expensive. To reduce this cost, we introduce a compact surrogate that compares each outside item only against the weakest top- k score. Let $s_i := \mathbf{w}^\top \mathbf{x}_i$ denote the score of item i , and define $z_{\min} := \min\{s_i : i \in \mathcal{T}\}$, where $\mathcal{T}$ is the current top- k set. For each outside item $j \notin \mathcal{T}$, we introduce a single slack variable $r_j \geq 0$ and impose $r_j \geq s_j - z_{\min}$.

Under this formulation, an outsider incurs a penalty only when its score exceeds the smallest score within the top- k set. This *minimum-among-top- k vs. outside* surrogate replaces k pairwise comparisons per outsider with one aggregated constraint, leading to a reduction in the total number of constraints and consequently may lead to a substantial improvement in solver runtime.

While this simplification effectively controls the aggregate outsider violation, it constitutes a relaxation of the original objective. Hence, it may admit slightly higher pairwise violations and, in turn, a minor degradation in the quality measures.

Effect on quality measure. This optimization acts as a bounded relaxation of the original objective. Let $r_{ij}^{\text{pair}} = \max\{0, s_j - s_i\}$ denote the original pairwise slack between an outside item j and a top- k item i . Since $s_i \geq z_{\min}$ for all $i \in \mathcal{T}$, we have $r_{ij}^{\text{pair}} \leq \max\{0, s_j - z_{\min}\} = r_j$, and therefore, $\sum_{i \in \mathcal{T}} r_{ij}^{\text{pair}} \leq k r_j$. Hence, the aggregate outsider-related loss in the original formulation is upper-bounded by k times the surrogate loss used here:

$$\sum_{j \notin \mathcal{T}} \sum_{i \in \mathcal{T}} r_{ij}^{\text{pair}} \leq \sum_{j \notin \mathcal{T}} \sum_{i \in \mathcal{T}} r_j = \sum_{i \in \mathcal{T}} \sum_{j \notin \mathcal{T}} r_j = k \sum_{j \notin \mathcal{T}} r_j.$$

This bound implies that minimizing the surrogate objective also reduces the true violation measure, though not necessarily to the same extent. In practice as discussed in Section 7, this relaxation can slightly increase the observed error metrics (e.g., PP or INV) but typically by a small margin relative to the substantial reduction in solver time.

(B) Incremental Top- k Constraints

The second optimization targets the large number of pairwise constraints among items within the top- k set. Instead of instantiating

all such constraints at once, we employ a *constraint generation* strategy that adds them incrementally based on violations observed in the current solution.

The process begins with a small *seed model* consisting of:

- **Tie-tier constraints** — for each group of tied items within top- k , we designate one item as a representative (the “hub”) and connect all others to it through equality constraints of the form $s_{\text{hub}} = s_j$. This creates a star-shaped constraint structure enforcing score equality within the tie group while avoiding the quadratic number of pairwise comparisons required in a full formulation.
- **Cross-tier neighbour constraints** — for items belonging to consecutive tiers in the desired top- k ranking, we introduce hinge constraints that preserve their intended order. Specifically, if an item i from a higher tier should outrank an item j from the immediately lower tier, we enforce $s_i \geq s_j$ through a hinge constraint of the form $e_{ij} \geq s_j - s_i$, with $e_{ij} \geq 0$. Each such constraint penalizes violations where a lower-tier item attains a higher score than its neighbour above it. By adding these local ordering relations between adjacent tiers, the model establishes an initial ordering backbone, which is later refined through incremental constraint generation.
- The constraints corresponding to Top k vs Outside.

The LP is then solved iteratively. After each solve, the scores of all top- k items are examined:

- (1) If a pair of items (i, j) from different tiers is found to violate the desired order—that is, an item j intended to rank below item i attains a higher score ($s_j - s_i > 0$)—we introduce a new hinge constraint to correct this inconsistency. Each such addition can be viewed as the LP “learning” a missing ordering relation—tightening the feasible region exactly where the current solution fails to respect the target ranking.
- (2) If two items within the same tier are found to have unequal scores, a new pair of constraints is introduced to enforce their equality by bounding the score difference in both directions.

This iterative process continues by repeatedly refining the constraint set based on violations observed in the current solution. With each iteration, additional constraints are introduced only where necessary, allowing the formulation to progressively approach the full model. If the process runs to termination — when no violated pairs remain — the resulting solution coincides with that of the complete inside-top- k formulation, achieving the **same optimal** objective value but with far fewer intermediate constraints.

Early-break variant. To further accelerate the process, we introduce an optional *early-break* mode that terminates constraint generation after a user-defined number of iterations. For instance, stopping after the first iteration enforces only the most critical violated pairs—those identified from the initial seed—while allowing minor residual violations to remain uncorrected. This strategy yields an approximate yet well-controlled relaxation of the full objective — as additional violated constraints are introduced, the objective monotonically increases, converging to the exact optimum when no violations remain. In practice (discussed in Section 7), the first iteration captures the majority of large violations, making the early-break solution a close numerical approximation of the full model while achieving substantial runtime savings.

Remark. These optimizations make the proposed approach an **anytime approach**: user can choose when to terminate (i.e. after how many iterations) based on the available computational budget. Additional iterations typically improve the solution quality but also increase the solving time. This adjustable termination criterion thus offers a practical knob to balance time and error measures, allowing our method to adapt gracefully to varying resource constraints.

7 EXPERIMENTAL EVALUATION

The experimental evaluation aims to compare our proposed algorithms with existing baselines in terms of scalability and quality of the synthesized utility functions, and to study the effect of the proposed optimization strategies. We first describe the experimental setup, baselines, and datasets used. The code base, extended results, and additional plots are available at <https://tinyurl.com/lin-rank>.

Setup. All experiments were conducted on a high-performance computing server equipped with an Intel Xeon w5-2465X CPU (24 cores, 3.1 GHz) and 96 GB RAM. A memory limit of 4 GB per core was enforced. Unless stated otherwise, all tools were executed in single-threaded mode on a single core, with a timeout of 3600 seconds per run. All implementations were written in Python 3.10 and used GUROBI 12.3 as the underlying optimization solver.

Baselines. We compared LP-PP (section 3.1), dP-PP (section 3.2), LS-INV (section 5.1), and HP-INV (section 5.2) with respect to scalability and quality analysis. Moreover, we also compared the following two optimization-based baselines that learn positive linear utility functions over the given feature space, (i) RankHow [13], which synthesizes simple linear utility functions that reproduce the ranking while minimizing a position-based error. (ii), Simplex-Constrained Linear Regression, this baseline fits a non-negative, L_1 -normalized weight vector using constrained least squares, aligning the linear scores with a monotonic target vector derived from the desired top- k order. Formally, it solves: $\min_{\alpha \geq 0, \mathbf{1}^\top \alpha = 1} \|X\alpha - y\|_2^2$, where X denotes the feature matrix and y is the target vector constructed from the top- k ranking. This is formulated as a convex quadratic program and solved using GUROBI [22].

Datasets. Different datasets used in our experimental evaluation are summarized in Table 3. Detailed discussion about the datasets is available in the extended version of the paper.

We aim to understand, across different error measures, the trade-off between scalability and solution quality for different proposed algorithms and aforementioned baselines. In particular, we sought to answer the following research questions:

- (1) For smaller and larger values of k , how well do different algorithms scale with increasing data size?
- (2) How does the quality of their solutions vary across different error measures?
- (3) Affiliation study with respect to runtime and quality for the proposed optimizations LP-based approaches?
- (4) Do the proposed optimizations make our approach scalable to large, real-world datasets?

For RQ1, RQ2, and RQ3, we used the NBA dataset to maintain consistency with prior work, and did additional experiments on synthetic dataset to analyze the effect of all proposed optimizations. For RQ4, we employed the JEE dataset.

The results can be summarized as follows: Regarding baselines, the LR baseline is the fastest but consistently yields the poorest solution quality across all error metrics. The current state-of-the-art, RANKHow, does not scale beyond $n = 1000$ and $k = 5$ on the NBA dataset and fails to produce results for other datasets and parameter settings; hence, we omit it from further discussion.

Among our methods, the optimized LP-PP variant provides the best trade-off between scalability and quality. On the NBA dataset with $n \approx 20,000$ and $k/n \geq 20\%$, it is up to **5× faster** than all methods except LR. Similar trends hold for the synthetic dataset, where LP-PP scales up to $k = 100$ with 100K points. For larger k , scalability degrades due to partially unobserved features and hidden additive bonuses, limiting performance beyond $\sim 50K$ points and $k \approx 1500$. For smaller k , HP-INV and dP-PP perform comparably to the unoptimized LP-PP in both quality and runtime; however, all three methods time out as k increases. On the JEE dataset, our methods scale to 250K points with k up to 20% (50K) within 1000 seconds, and further improve to under 500 seconds with early termination.

Plots: We present detailed plots illustrating these findings, with additional results in the appendix. In all plots, the x -axis shows the values of k , while the y -axis reports the corresponding error measures (PP, INV, MPE, LIPE); a separate plot shows runtime.

We evaluate LP-PP under three configurations: (i) LP-PP (OVM : 0, INC : 0) (unoptimized); (ii) LP-PP (OVM : 1, INC : 0), which compares the minimum element in the top- k set against elements outside it (Section 6 A); and (iii) LP-PP (OVM : 1, INC : 1, EB : 1), an incremental variant with early termination (Section 6 B)².

Scalability vs Quality Trade-off (RQ1 and RQ2). We first discuss results on the NBA dataset (Figure 4). For $n = 1000$, LR and HP-INV show higher errors even for small k . As k increases, LR degrades rapidly, while HP-INV and LS-INV fail to scale beyond $k \approx 100$. In contrast, LP-PP and dP-PP perform similarly in both runtime and error for smaller datasets. For $n = 10,000$, dP-PP and LS-INV fail to scale for larger k , often hitting the 3600s timeout, whereas LP-PP remains within a few hundred seconds. Overall, LP-PP achieves the best balance between scalability and quality as n and k increase. While dP-PP and LS-INV are comparable to LP-PP for small n , they do not scale to larger settings.

We now discuss the synthetic dataset (Figure 5). For $n = 10,000$, the overall trends are similar to those observed on the NBA dataset. As k increases, LP-PP scales well and remains within a few hundred seconds. In contrast, dP-PP, LS-INV and HP-INV struggle to scale for larger k . Although LR scales efficiently, it performs the worst across all error measures.

A key observation is the impact of hidden additive bonuses in the true scoring function. Due to these unobserved factors, LS-INV also performs poorly across most error metrics. In comparison, LP-PP and dP-PP achieve similar performance across all measures. While INV scores are relatively similar across methods, other metrics (PP, MPE, LIPE) clearly show the weaker performance of LR and LS-INV. As we increase the dataset size to $n = 50,000$, scalability becomes more challenging. dP-PP times out even for small values of k . LP-PP

²The earlybreak flag controls the stopping criterion (number of iterations). More iterations improve PP at the cost of higher runtime, making the method anytime.

Dataset	#n	#d	Ranking	Processing Details
NBA [1]	22,840	8	Predefined utility function	Real-world player statistics; all features are min-max normalized; ranking follows a standard utility scheme from prior work
Synthetic	50,000	7	Linear score + additive bonuses	Features (e.g., relevance, popularity, freshness) are sampled from statistical distributions and combined via a linear function. Two small hidden effects are added: a boost for fresh/sponsored items and a diversity penalty for frequent categories. Final ranking is based on this adjusted score
JEE 2009 [9]	~384,000	5	Total score + additive bonuses	Raw marks in Mathematics, Physics, and Chemistry are combined with categorical attributes (category and gender). Additive bonuses are applied (ST +40, SC +30, OBC +15, GEN +0; Female +10, Male +0), and the final ranking is based on the adjusted total score

Table 3: Summary of datasets used in our experiments.

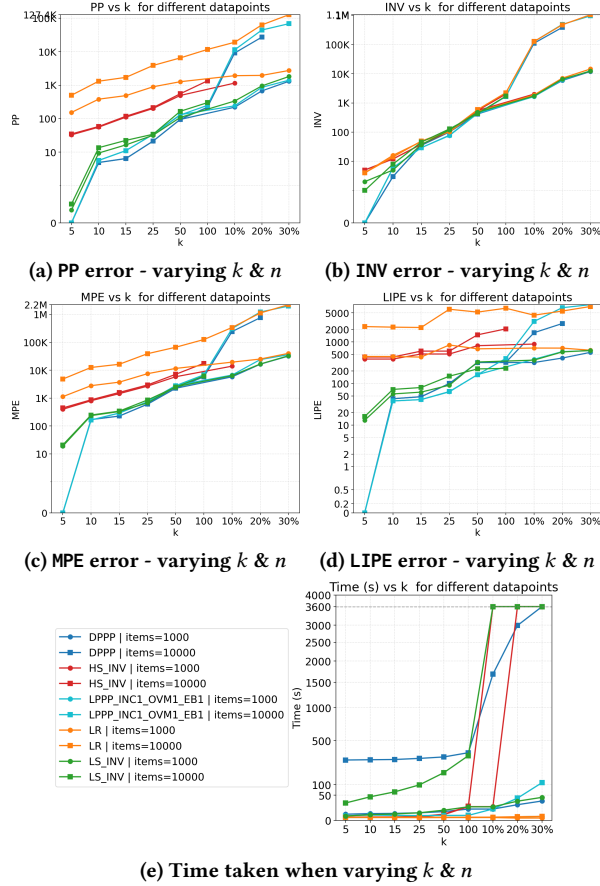


Figure 4: [NBA dataset] For $n = 1000, 10000$ and varying values of k , these subplots show the different error measures and time taken. The approaches that timed out do not have their corresponding points shown in the error measure plots. $x\%$ on axis represents the $k = x\%$ of n .

continues to scale better but begins to struggle beyond $k = 5000$, and eventually times out at $k = 15,000$.

Although dP-PP is expected to improve PP at higher cost, in practice it achieves similar PP as LP-PP but with higher runtime. Finally, HP-INV scales poorly due to the overhead of merging quadtree partitions, which becomes a bottleneck for large k .

Affiliation study for LP based approaches (RQ3). Comparing the LP-PP variants, we observe that incremental solving with early termination provides the best balance between scalability and solution quality across all error measures. As shown in Figure 6, for NBA dataset with $n = 10000$, the fully optimized variant, LP-PP (ovm : 1, inc : 1, eb : 1), solves instances in under 100 seconds,

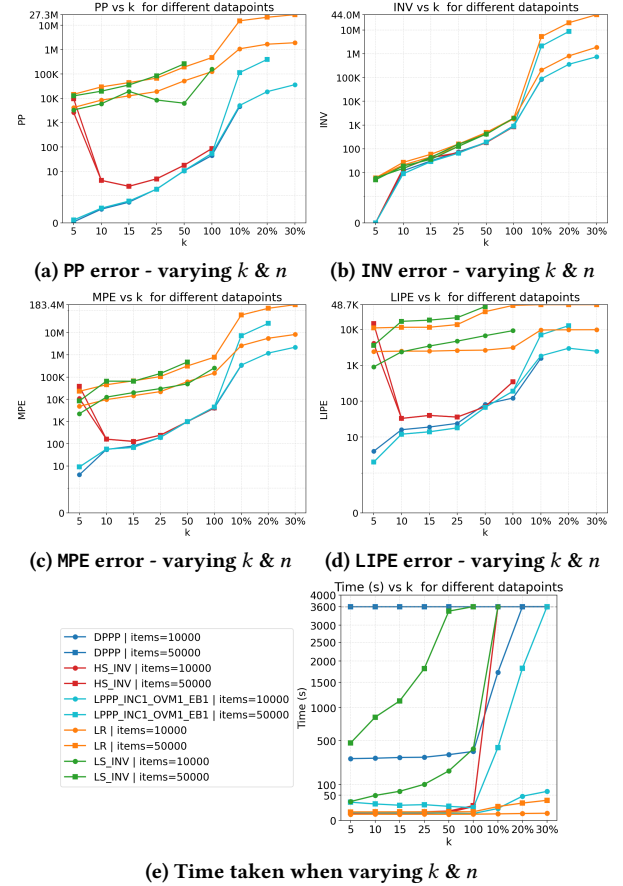


Figure 5: [Synthetic dataset] For $n = 10000, 50000$ and varying values of k , these subplots show the different error measures and time taken. The approaches that timed out do not have their corresponding points shown in the error measure plots. $x\%$ on axis represents the $k = x\%$ of n .

whereas the unoptimized version times out as k exceeds 1000. This improvement comes at a modest cost of about $2.5\times$ increase in PP error, with similar trends observed across other error measures. Overall, this variant offers the highest scalability with only limited degradation in quality. The next best configuration is LP-PP (ovm : 1, inc : 0), which achieves slightly better error values but is significantly slower-up to $5-12\times$ for larger instances. Thus, it provides higher solution quality at the expense of scalability.

Similar trends are observed on the synthetic datasets; detailed results are provided in the extended version of the paper.

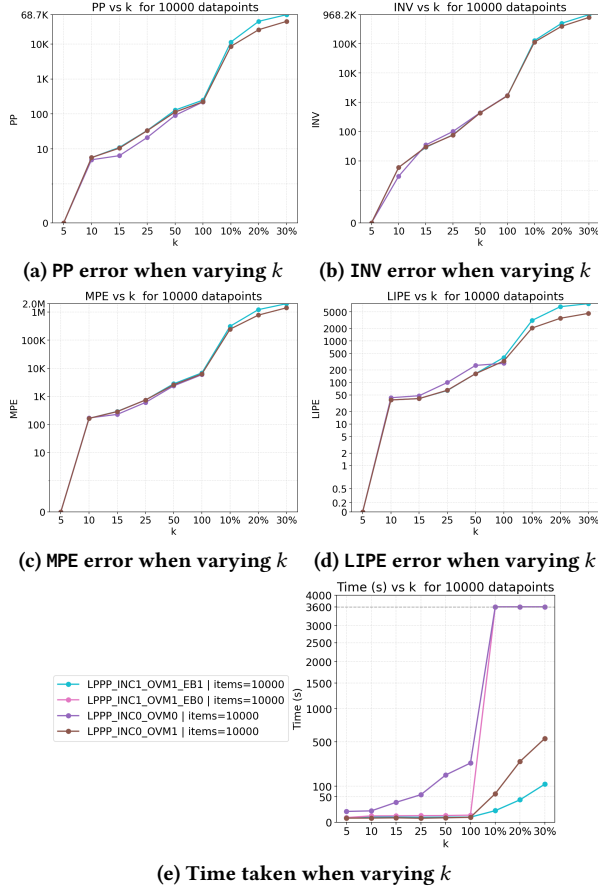


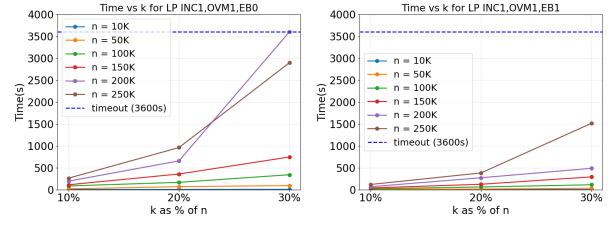
Figure 6: [NBA dataset] For $n = 10000$ and varying values of k , these subplots show different error measures and time taken for different variants of LP-PP.

Scalability with respect to real world larger datasets (RQ4). For the JEE dataset, we evaluated scalability by varying the dataset size n from 10000 to 250000 and the top- k fraction as 10%, 20%, and 30% of n . As shown in Figure 7, we compared the best-performing variant, LP-PP (OVM : 1, INC : 1, EB : 1), with the second-best variant, LP-PP (OVM : 1, INC : 1, EB : 0). Other baseline approaches, such as RANKHOW, could not scale beyond $(n, k) = (1000, 5)$, while LR scaled but yielded high error measures. Hence, we did not include them in this experiment.

Both LP-PP variants considered demonstrate strong scalability across all dataset sizes. For $k = 20\%$, both configurations remain efficient across all values of n , requiring at most 500 s and 1000 s, respectively. For $k = 30\%$, at $n = 250,000$, the second-best variant reaches the timeout limit, whereas the optimized LP-PP completes in approximately 1500 s. These results demonstrate that our proposed LP-based approaches are highly scalable and practical for large real-world datasets.

8 RELATED WORK

Top- k in utility functions: Many classes of utility functions have been studied in the literature. Monotone utility functions [16] form a class of utility functions, which, when given two tuples t_i and t_j such that $t_i[x] > t_j[x]$ in every attribute x , would score t_i better than t_j . Quickly determining Top- k over the class of monotone functions was studied by Fagin et al. [17] (Threshold and Fagin’s



(a) Time taken by LP-PP with (b) Time taken by LP-PP with
(INC : 1, OVM : 1, EB : 0) (INC : 1, OVM : 1, EB : 1)
Figure 7: Scalability evaluation for JEE dataset for different LP variants.

algorithm). Xin et al. [43] study the problem of quickly determining the top- k over user preferences and a subset of attributes, which were formulated as convex functions. Non-linear functions, like quadratic functions [27] and polynomial functions [32] have been studied in the context of multi-criteria top- k query. An interesting class of functions that have been widely studied is linear functions which have been used in multi-criteria decision making [30], utility/ranking [41, 42], explaining complex Machine Learning models [33]. While non-linear, monotone and other classes of functions have been studied in the context of generating top- k , utility and scoring based works [4, 35] have considered the class of linear functions as models of explanations in our work.

Learning the ranking function. Chen et al. [13] initiated the study of explaining rankings via linear utility functions and introduced four error measures as part of their explainability framework. However, their analysis focuses primarily on the MPE objective, and a comparative or unified treatment of the remaining measures is absent. In contrast, we develop a unifying perspective that studies formal relationships among all four error measures and propose algorithmic techniques, including heuristic methods applicable across multiple objectives.

Classical regression techniques-such as linear models [28], decision trees [11], and neural networks [7], assume access to numerical labels and are thus inapplicable when only a ranking is provided. Nonetheless, we include linear regression as a baseline for comparison. Model-agnostic explanation approaches such as LIME [33] and its variants [20], as well as methods proposed in information retrieval [40], rely on querying the underlying model through synthetic sampling, which is infeasible in our setting where the ranking function is unknown and cannot be accessed as an oracle. *Our work fills this gap by systematically comparing error measures and developing scalable optimization techniques that operate directly on rankings without requiring score access or model queries.*

9 DISCUSSION AND FUTURE WORK

In this work, we look at explanations to ranking problems based on linear utility/ranking functions. An interesting direction one could consider are to extend our methods, or design new methods to other classes of ranking functions like quadratic/non-linear scoring functions, or a more generic class of functions like monotone functions. Another promising direction to consider is to develop efficient approaches to LIPE, which is a “worst case” error measure (ℓ_∞ measure) over the set of rank differences. We plan to consider both these exciting directions as extensions in future work.

REFERENCES

- [1] 2024. *Calculating Player Efficiency Rating (PER)*. <https://www.basketball-reference.com/about/per.html>
- [2] Josh Alman and Virginia Vassilevska Williams. 2024. A refined laser method and faster matrix multiplication. *TheoretCS* 3 (2024), 32.
- [3] Sanjeev Arora. 1998. Polynomial time approximation schemes for Euclidean traveling salesman and other geometric problems. *Journal of the ACM (JACM)* 45, 5 (1998), 753–782.
- [4] Abolfazl Asudeh, Hosagrahar Visvesvaraya Jagadish, Julia Stoyanovich, and Gautam Das. 2019. Designing Fair Ranking Schemes. In *Proceedings of the 2019 International Conference on Management of Data*. ACM, Amsterdam, Netherlands, 1259–1276.
- [5] Abolfazl Asudeh, Azade Nazi, Nan Zhang, Gautam Das, and HV Jagadish. 2019. RRR: Rank-regret representative. In *Proceedings of the 2019 International Conference on Management of Data*. ACM, New York, NY, USA, 263–280.
- [6] Solon Barocas and Andrew D Selbst. 2016. Big data’s disparate impact. *Calif. L. Rev.* 104 (2016), 671.
- [7] Joshua Bassey, Lijun Qian, and Xianfang Li. 2021. A Survey of Complex-Valued Neural Networks. <https://arxiv.org/abs/2101.12249>. arXiv:2101.12249.
- [8] Emery Berger. 2025. *CSRanking*. <https://csrankings.org/#/index?all&us>
- [9] Guangya Cai. 2025. Finding a Fair Scoring Function for Top-k Selection: From Hardness to Practice. <https://arxiv.org/abs/2503.11575>. arXiv:2503.11575.
- [10] L. Elisa Celis, Damian Straszak, and Nisheeth K. Vishnoi. 2017. Ranking with Fairness Constraints. <https://arxiv.org/abs/1704.06840>. , 27 pages. arXiv:1704.06840.
- [11] Bahzad Charbuty and Adnan Abdulazeez. 2021. Classification based on decision tree algorithm for machine learning. *Journal of applied science and technology trends* 2, 01 (2021), 20–28.
- [12] Zixuan Chen, Panagiotis Manolios, and Mirek Riedewald. 2023. Why not yet: Fixing a top-k ranking that is not fair to individuals. *Proceedings of the VLDB Endowment* 16, 9 (2023), 2377–2390.
- [13] Zixuan Chen, Panagiotis Manolios, and Mirek Riedewald. 2025. Synthesizing Scoring Functions for Rankings Using Symbolic Gradient Descent. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, Piscataway, NJ, USA, 3849–3862.
- [14] Michael B Cohen, Yin Tat Lee, and Zhao Song. 2021. Solving linear programs in the current matrix multiplication time. *Journal of the ACM (JACM)* 68, 1 (2021), 1–39.
- [15] Daniel Dadush and Sophie Huiberts. 2018. A friendly smoothed analysis of the simplex method. In *Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing*. Association for Computing Machinery, New York, NY, USA, 390–403.
- [16] Ronald Fagin. 2002. Combining fuzzy information: an overview. *ACM SIGMOD Record* 31, 2 (2002), 109–118.
- [17] Ronald Fagin, Amnon Lotem, and Moni Naor. 2001. Optimal aggregation algorithms for middleware. In *Proceedings of the Twentieth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS)*. ACM, New York, NY, USA, 102–113.
- [18] Taylor Kessler Faulkner, Will Brackenbury, and Ashwin Lall. 2015. k-Regret Queries with Nonlinear Utilities. *Proc. VLDB Endow.* 8, 13 (2015), 2098–2109.
- [19] Abraham Gale and Amélie Marian. 2020. Explaining monotonic ranking functions. *Proceedings of the VLDB Endowment* 14, 4 (2020), 640–652.
- [20] Damien Garreau and Ulrike Luxburg. 2020. Explaining the explainer: A first theoretical analysis of LIME. In *International conference on artificial intelligence and statistics*. PMLR, Proceedings of Machine Learning Research, Cambridge, MA, USA, 1287–1296.
- [21] Qianyu Guo, Haotong Gong, Xujun Wei, Yanwei Fu, Yizhou Yu, Wenqiang Zhang, and Weifeng Ge. 2023. RankDNN: Learning to Rank for Few-Shot Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press, Washington, DC, USA, 728–736.
- [22] Gurobi Optimization, LLC. 2024. Gurobi Optimizer Reference Manual. <https://www.gurobi.com>
- [23] Ihab F Ilyas, George Beskales, and Mohamed A Soliman. 2008. A survey of top-k query processing techniques in relational database systems. *ACM Computing Surveys (CSUR)* 40, 4 (2008), 1–58.
- [24] William B. Johnson and Joram Lindenstrauss. 1984. Extensions of Lipschitz mappings into a Hilbert space. *Contemp. Math.* 26 (1984), 189–206.
- [25] Ian T. Jolliffe. 2002. *Principal Component Analysis*. Springer, New York.
- [26] Adam Kalai and Santosh Vempala. 2005. Efficient algorithms for learning linear separators under log-concave distributions. *J. Comput. System Sci.* 71, 3 (2005), 291–307.
- [27] Taylor Kessler Faulkner, Will Brackenbury, and Ashwin Lall. 2015. K-regret queries with nonlinear utilities. *Proceedings of the VLDB Endowment* 8, 13 (2015), 2098–2109.
- [28] Dastan Maulud and Adnan M Abdulazeez. 2020. A review on linear regression comprehensive in machine learning. *Journal of applied science and technology trends* 1, 2 (2020), 140–147.
- [29] Marco Morik, Ashudeep Singh, Jessica Hong, and Thorsten Joachims. 2020. Controlling fairness and bias in dynamic learning-to-rank. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, New York, NY, USA, 429–438.
- [30] Danupon Nanongkai, Atish Das Sarma, Ashwin Lall, Richard J Lipton, and Jun Xu. 2010. Regret-minimizing representative databases. *Proceedings of the VLDB Endowment* 3, 1-2 (2010), 1114–1124.
- [31] United Nation. 2025. *Sustainable Development Report Ranking*. <https://dashboards.sdgindex.org/rankings/>
- [32] Jianzhong Qi, Fei Zuo, Hanan Samet, and Jia Cheng Yao. 2018. K-regret queries using multiplicative utility functions. *ACM Transactions on Database Systems (TODS)* 43, 2 (2018), 1–41.
- [33] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. Why Should I Trust You? Explaining the Predictions of Any Classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, New York, NY, USA, 1135–1144.
- [34] Procheta Sen, Debasis Ganguly, Manisha Verma, and Gareth JF Jones. 2020. The curious case of IR explainability: Explaining document scores within and across ranking models. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, New York, NY, USA, 2069–2072.
- [35] Suraj Shetiya, Abolfazl Asudeh, Sadia Ahmed, and Gautam Das. 2019. A unified optimization algorithm for solving “regret-minimizing representative” problems. *Proceedings of the VLDB Endowment* 13, 3 (2019), 321–334.
- [36] Daniel A Spielman and Shang-Hua Teng. 2004. Smoothed analysis of algorithms: Why the simplex algorithm usually takes polynomial time. *Journal of the ACM (JACM)* 51, 3 (2004), 385–463.
- [37] V Srinivasan. 1976. Linear programming computational procedures for ordinal regression. *Journal of the ACM (JACM)* 23, 3 (1976), 475–487.
- [38] Thibaut Thonet, Yagmur Gizem Cinar, Eric Gaussier, Minghan Li, and Jean-Michel Renders. 2022. Listwise learning to rank based on approximate rank indicators. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press, Washington, DC, USA, 8494–8502.
- [39] Pravin M. Vaidya. 1989. Speeding-Up Linear Programming Using Fast Matrix Multiplication. In *Proceedings of the 30th Annual Symposium on Foundations of Computer Science*. IEEE, Research Triangle Park, NC, USA, 332–337.
- [40] Manisha Verma and Debasis Ganguly. 2019. LIRME: Locally Interpretable Ranking Model Explanation. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, Paris, France, 1281–1284.
- [41] Weicheng Wang, Raymond Chi-Wing Wong, Jinyang Li, and H. V. Jagadish. 2025. Interactive Learning for Diverse Top-k Set. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, Utrecht, Netherlands, 3084–3097.
- [42] Weicheng Wang, Raymond Chi-Wing Wong, and Min Xie. 2021. Interactive Search for One of the Top-k. In *Proceedings of the 2021 International Conference on Management of Data*. ACM, Virtual Event, 1920–1932.
- [43] Dong Xin, Jiawei Han, Hong Cheng, and Xiaolei Li. 2006. Answering top-k queries with multi-dimensional selections: The ranking cube approach. In *Proceedings of the 32nd International Conference on Very Large Data Bases (VLDB)*. VLDB Endowment, Seoul, Korea, 463–475.
- [44] Jun Xu and Hang Li. 2007. Adarank: a boosting algorithm for information retrieval. In *Proceedings of the 30th annual international ACM SIGIR conference on Research and development in information retrieval*. Association for Computing Machinery, New York, NY, USA, 391–398.

A MISSING PROOFS OF SECTION 3

THEOREM 3.2 *Given a database $\mathcal{D}$ and a ranking order π , let OPT^* denote the optimal PP error. Then dP-PP returns a solution with error at most $\frac{1}{\sqrt{d^2 - 2\sqrt{d}(d-1)}} \cdot \text{OPT}^*$, and runs in time $O(d(n^2 + d)^{2.5}L)$, where L is the number of bits required to represent the input.*

PROOF. Similar to the previous proof, we first analyze the quality of the solution returned by the dP-PP method, and then discuss its running time.

Approximation analysis: We begin by analyzing the approximation guarantee for a single region among the d partitions. Since all regions are symmetric, the same bound applies to the others.

Consider the hyperplane defined by the pivot point together with a set of $d - 1$ corner points as follows:

$$\begin{bmatrix} 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & 0 \\ \frac{1}{\sqrt{d}} & \frac{1}{\sqrt{d}} & \cdots & \frac{1}{\sqrt{d}} & \frac{1}{\sqrt{d}} \end{bmatrix} \quad (9)$$

Similar to the proof of Theorem 3.1, our goal is to identify the vector within the region for which the ratio between its $\|W\|_1$ value and its $\|W\|_2$ norm is minimized. Since each vector corresponding to OPT has $\|W\|_2 = 1$, this reduces to finding the vector in the region whose $\|W\|_1$ value is minimized. To compute this quantity, we rely on geometric arguments.

As a first step, we compute the equation of the plane $\sum_{i=1}^d a_i w_i = c$ corresponding to the set of points in Equation 9. From the first $d - 1$ points we obtain $a_1 = a_2 = \cdots = a_{d-1} = c$. From the last point,

$$a_1 \frac{1}{\sqrt{d}} + a_2 \frac{1}{\sqrt{d}} + \cdots + a_{d-1} \frac{1}{\sqrt{d}} + a_d \frac{1}{\sqrt{d}} = c.$$

Substituting $a_1 = a_2 = \cdots = a_{d-1} = c$ into the above,

$$(d-1)c \frac{1}{\sqrt{d}} + a_d \frac{1}{\sqrt{d}} = c,$$

which simplifies to $a_d = -c(d-1-\sqrt{d})$. Normalizing by c (since plane equations are defined up to scale), the final equation of the plane is

$$w_1 + w_2 + \cdots + w_{d-1} - (d-1-\sqrt{d})w_d = 1. \quad (10)$$

We now perform the final computation. Since each vector W corresponding to OPT has unit length, the vector on the hyperplane (10) with the smallest ℓ_2 norm determines the approximation error. Equivalently, this is the minimum Euclidean distance from the origin to the hyperplane.

For a hyperplane $\sum_{i=1}^d a_i x_i = c$ and a point $(t_1, \dots, t_d)$, the distance from the point to the hyperplane is $\frac{|\sum_{i=1}^d a_i t_i - c|}{\sqrt{\sum_{i=1}^d a_i^2}}$. Hence, the distance from the origin $(0, \dots, 0)$ to (10) equals

$$\begin{aligned} &= \frac{1}{\sqrt{\underbrace{1^2 + \cdots + 1^2}_{d-1 \text{ terms}} + (d-1-\sqrt{d})^2}} \\ &= \frac{1}{\sqrt{(d-1) + (d-1-\sqrt{d})^2}} \\ &= \frac{1}{\sqrt{d^2 - 2\sqrt{d}(d-1)}}. \end{aligned}$$

Therefore, our solution achieves at least $\frac{1}{\sqrt{d^2 - 2\sqrt{d}(d-1)}} \cdot \text{OPT}^*$.

Runtime analysis: The dP-PP method invokes the LP-PP procedure d times, once for each partition. Therefore, its overall running time is d times that of LP-PP, yielding a total complexity of $O(d(n^2 + d)^{2.5}L)$. $\square$

THEOREM 3.3 *Given a database $\mathcal{D}$ and a ranking order π , let OPT^* denote the optimal PP error. Then the GPP method returns a solution with error at most $(1+\epsilon) \text{OPT}^*$ and runs in time $O\left(\frac{(\pi e/2)^{d/2}}{\epsilon^d} (n^2 + d)^{2.5}\right)$.*

PROOF. Each of the δ -hypercubes on which the modified LP (Equation 5) is run represents a region of the space where the error must be evaluated. The maximum possible error is proportional to the ratio between the full vector length and the portion covered by the LP, i.e., the part of the length accounted for by the completely contained grid cubes. For our analysis, we consider only those hypercubes that are *near* the surface of the d -sphere but lie within it. This is sufficient, since any hypercube located in the same direction but closer to the origin would yield a much looser bound than one near the surface.

Now consider a δ -hypercube that intersects the surface along an arbitrary vector. The portion of this hypercube that lies below the surface is fully contained within the d -sphere, and hence the LP can be evaluated on it. The deviation of the vector corresponding to this hypercube from the actual surface point is bounded by the diagonal length of the hypercube, which is $\delta\sqrt{d}$. Therefore, the error is at most $\frac{1}{1-\delta\sqrt{d}}$ times the optimal value on the corresponding point of the d -sphere.

For a $(1+\epsilon)$ -approximation scheme, we require the approximation factor to be $1+\epsilon$. Solving for the corresponding δ gives $\delta = \frac{\epsilon}{\sqrt{d}(1+\epsilon)}$.

Runtime analysis: To compute the number of cubes on which we need to compute the LP, we estimate the ratio of the volume of the d -sphere to that of the volume of δ -hypercube. Hence, there will be at most $\left\lceil \frac{\pi^{\frac{d}{2}}}{2^d \Gamma(\frac{d}{2}+1) \delta^d} \right\rceil$ many cubes completely inside the surface and running LP on each cube will cost $O((n^2 + d)^{2.5}L)$ time, where L is the number of bits needed to express the input. Hence, total

running time complexity will be $O\left(\left\lceil \frac{\pi^{\frac{d}{2}}}{2^d \Gamma(\frac{d}{2}+1) \delta^d} \right\rceil (n^2 + d)^{2.5} L\right)$.

$$\text{Runtime} = \begin{cases} O\left(\left\lceil \frac{\pi^{\frac{d}{2}}}{2^d \left(\frac{d}{2}\right)! \delta^d} \right\rceil (n^2 + d)^{2.5} L\right), & \text{if } d \text{ is even} \\ O\left(\left\lceil \frac{\pi^{\frac{d}{2}}}{2^d \left(\frac{d}{2}\right) \left(\frac{d}{2}-1\right) \cdots \left(\frac{1}{2}\right) \pi^{\frac{1}{2}} \delta^d} \right\rceil (n^2 + d)^{2.5} L\right), & \text{if } d \text{ is odd} \end{cases}$$

In terms of ϵ , worst case running time = $O\left(\left\lceil \frac{(\pi d)^{\frac{d}{2}} (1+\epsilon)^d}{(2\epsilon)^d \left(\frac{d}{2}\right)!} \right\rceil (n^2 + d)^{2.5} L\right)$. Using Stirling's approximation and further simplifying, we get

$$O\left(\frac{(2\pi e)^{\frac{d}{2}}}{(2\epsilon)^d d^{\frac{1}{2}}}\right) (n^2 + d)^{2.5} L = O\left(\frac{(\pi e/2)^{d/2}}{\epsilon^d}\right) (n^2 + d)^{2.5}$$

□

B MISSING PROOFS OF SECTION 4

LEMMA 1 For any input ranked order π and the ranked order produced by utility function f_W , $\text{INV} \leq \text{MPE} \leq 2 \times \text{INV}$

PROOF. $\text{MPE} \leq 2 \times \text{INV}$: We prove this by induction on the number of inversions INV :

Basis step: $\text{INV} = 1$. This happens only when the immediate neighboring tuples have been interchanged. In that case, $\text{MPE} = 2$ as the ranking of all other tuples will remain the same.

Induction step: Suppose for $\text{INV} = r$, $\text{MPE} \leq 2 \times r$. For $\text{INV} = r + 1$, immediately ranked tuples have to be inverted for going from r inversions to $r + 1$ inversions and hence

$$\text{MPE} \leq 2 \times r + 2 = 2 \times (r + 1) = 2 \times \text{INV}$$

$\text{INV} \leq \text{MPE}$: From definition;

- Inversions (INV)

$$|\{r \in R_\pi(k), s \in R, \pi(r) < \pi(s) \mid \rho_W(r) > \rho_W(s)\}| \quad (11)$$

- Minimum position error (MPE)

$$\sum_{r \in R_\pi(k)} |\rho_W(r) - \pi(r)| \quad (12)$$

Now suppose we have an inversion pair (r, s) such that $\pi(r) < \pi(s)$ & $\rho_W(r) > \rho_W(s)$. We show that for each such pair (r, s) , we will get atleast one tuple which will contribute a minimum of one to the MPE and since in MPE we are taking the sum of non negative value, the required result will follow.

For each inversion pair (r, s) , the possible subcases are as follows:

- (1) $\rho_W(s) < \rho_W(r) = \pi(r) < \pi(s)$: In this case, tuple s will contribute at-least one to the MPE.
- (2) $\rho_W(s) < \rho_W(r) < \pi(r) < \pi(s)$: In this case, tuple r will contribute at-least one to the MPE.
- (3) $\rho_W(s) < \rho_W(r) > \pi(r) < \pi(s)$: In this case, tuple r will contribute at-least one to the MPE.

As in each case, the contribution is at-least 1, $\text{INV} \leq \text{MPE}$. □

C MISSING PROOFS AND PSEUDO-CODES OF SECTION 5

THEOREM 5.4 (TIME COMPLEXITY) The overall running time of the hierarchical partitioning algorithm is $O(n^3(n+d)^{2.5}L)$, where L denotes the number of bits required to encode the input.

Algorithm 3: GMC: Greedy Merging of Cells

Input: Database $\mathcal{D}$, Ranking π , Ordered ranking L_1 , Ordered ranking L_2

Output: Best ranking

```

1 Initialize  $bestError \leftarrow \infty$ ,  $bestRanking \leftarrow \text{null}$ 
2 foreach tuple  $t \in L_2$  do
3   foreach order where we insert  $t$  into  $L_1$  do
4     // There are  $|L_1| + 1$  such orders
5     Add new constraints and solve LP for feasibility
6     if LP is feasible and error <  $bestError$  then
7        $bestError \leftarrow \text{error}$ 
7        $bestRanking \leftarrow \text{updated ranking}$ 
8 return  $bestRanking$ 
```

Algorithm 4: HP-INV : Hierarchical Partitioning for Inversions

Input: Database $\mathcal{D}$, ranking π

Output: Realizable order ρ

```

1  $T \leftarrow \text{ShiftedQuadTree}(\mathcal{D})$ 
2 foreach leaf node  $v$  in  $T$  do
3    $v.\text{order} \leftarrow t$  //  $t$  is the tuple in  $\mathcal{D}$  stored in  $v$ 
4 while there exists some internal node with unmerged children do
5   foreach internal node  $p$  in  $T$  (bottom-up) do
6     Let  $C_1, C_2, \dots, C_m$  be the child nodes of  $p$ 
7      $M \leftarrow C_1.\text{order}$ 
8     for  $i \leftarrow 2$  to  $m$  do
9        $S \leftarrow \text{Set}(M \cup C_i.\text{order})$ 
10       $O_1 \leftarrow \text{GMC}(M, C_i.\text{order})$ 
11       $O_2 \leftarrow \text{GMC}(C_i.\text{order}, M)$ 
12      Store  $O_1$  in  $M$  if  $\text{INV}(O_1, \pi_M) \leq \text{INV}(O_2, \pi_M)$  else
13        store  $O_2$  in  $M$ 
13    $p.\text{order} \leftarrow M$ 
14 return feasible order  $T.\text{root}.\text{order}$ 
```

PROOF. Each LP solved during merging contains $O(n+d)$ variables and constraints, and thus requires $O((n+d)^{2.5}L)$ time. Across the hierarchy, at most $O(n)$ merge operations are performed. In each merge, inserting elements from one list into the other involves testing up to $O(n^2)$ candidate positions, each requiring an LP solve. Therefore, the total runtime is $O(n^2 \cdot n \cdot (n+d)^{2.5}L) = O(n^3(n+d)^{2.5}L)$. □

D EXPERIMENTAL PLOTS

E CONCLUSION

We propose different LP based methods for the PP error which provide a trade-off between runtime and quality. Our work provides guarantees on both runtime and quality for these methods. We formulated an ILP for the INV problem and via relaxation of the Boolean constraints, show the interconnectedness among the seemingly different error measures. We also design two efficient approaches (hill-climbing and hierarchical partitioning HP-INV)

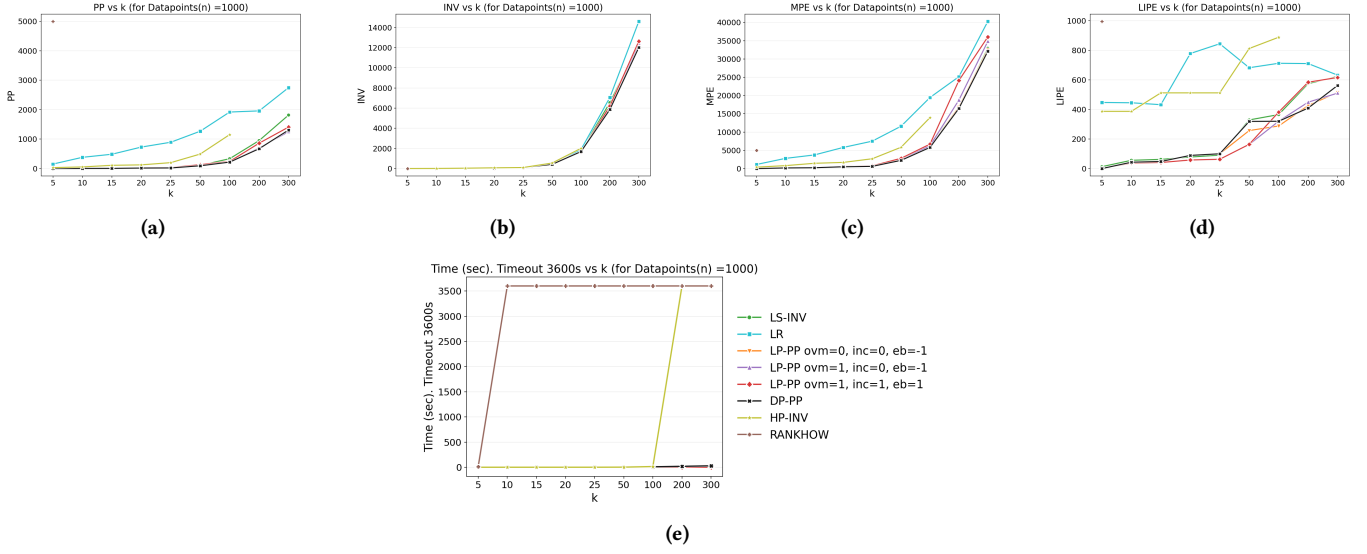


Figure 8: For $n = 1000$ and varying values of k , these subplots show different error measures and the time taken for the listed approaches. Approaches that timed out ($> 3600s$) have their points omitted in the error measure plots.

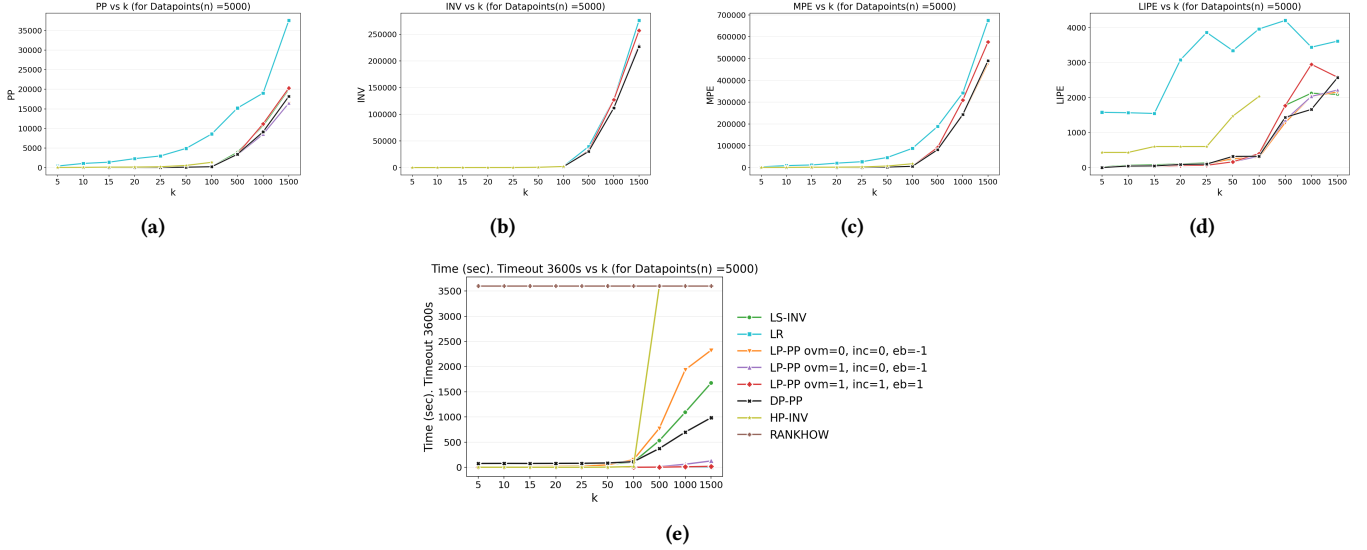


Figure 9: For $n = 5000$ and varying values of k , these subplots show different error measures and the time taken for the listed approaches. Approaches that timed out ($> 3600s$) have their points omitted in the error measure plots.

for the INV error measure. Finally, our extensive experiments on

real datasets demonstrate the efficacy, efficiency and scalability of our methods.

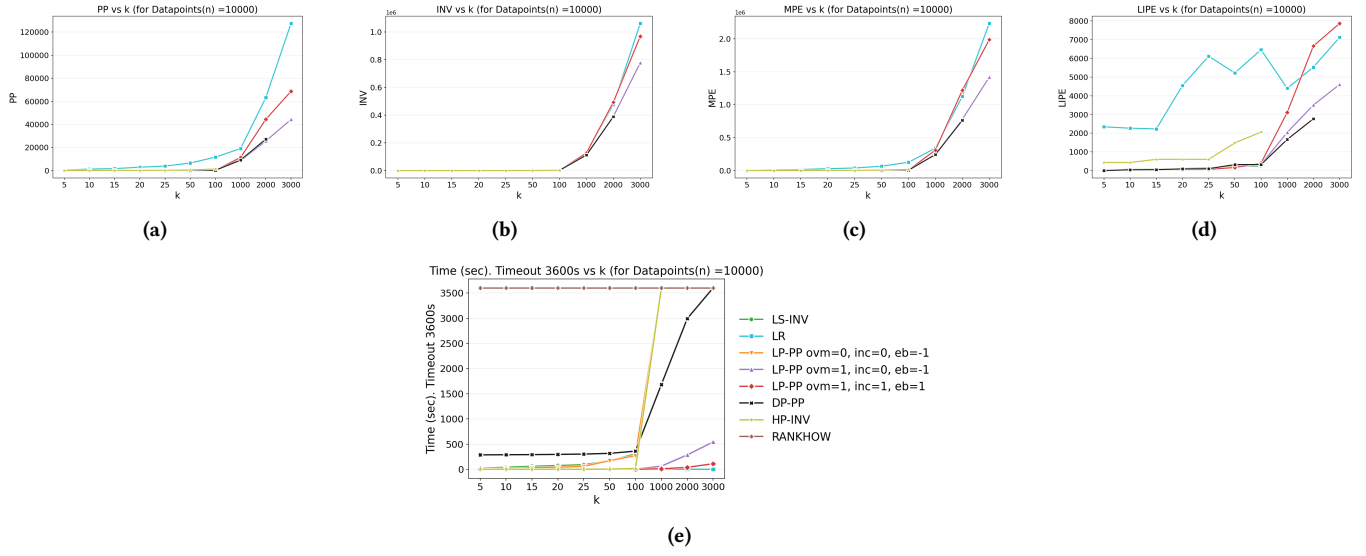


Figure 10: For $n = 10000$ and varying values of k , these subplots show different error measures and the time taken for the listed approaches. Approaches that timed out ($> 3600s$) have their points omitted in the error measure plots.

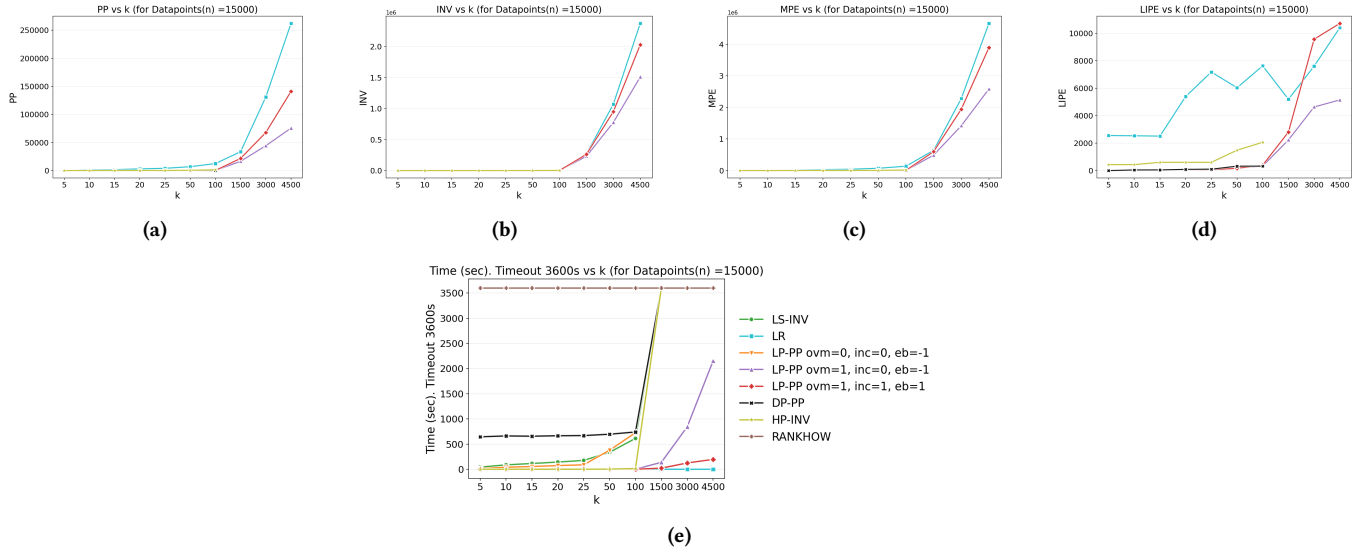


Figure 11: For $n = 15000$ and varying values of k , these subplots show different error measures and the time taken for the listed approaches. Approaches that timed out ($> 3600s$) have their points omitted in the error measure plots.

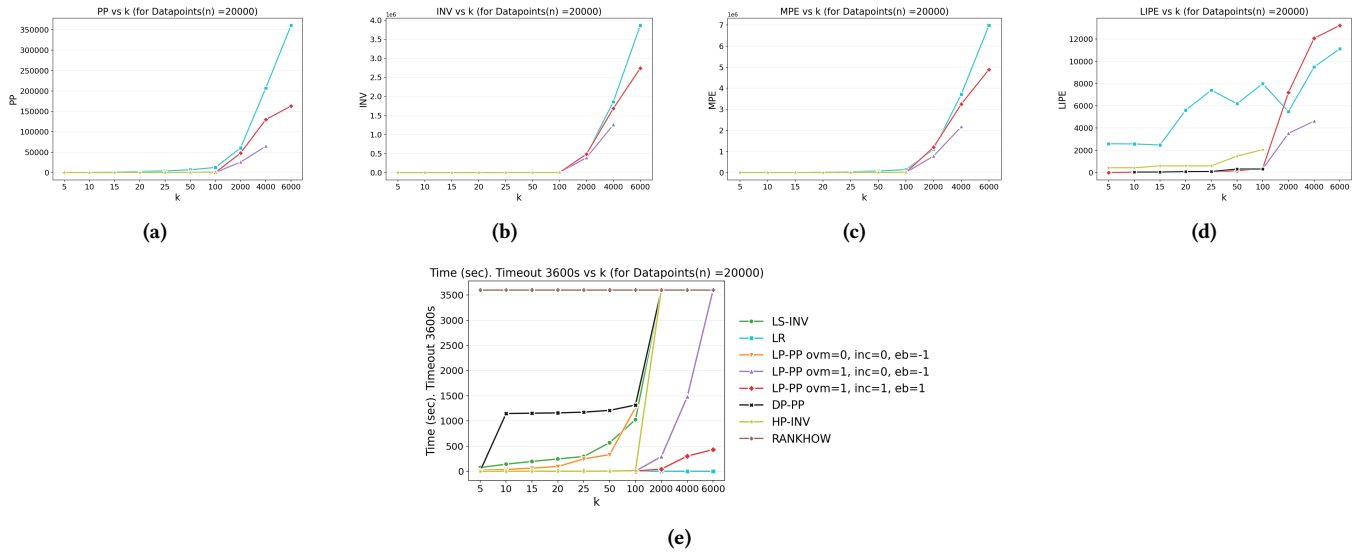


Figure 12: For $n = 20000$ and varying values of k , these subplots show different error measures and the time taken for the listed approaches. Approaches that timed out ($> 3600s$) have their points omitted in the error measure plots.